

RHEINISCH-WESTFÄLISCHE TECHNISCHE HOCHSCHULE AACHEN
Knowledge-Based Systems Group
Prof. G. Lakemeyer, Ph.D.

Multi-Agent System for Production Processes in the Automotive Industry

Bachelor's Thesis

Maximilian Kerpen
April 10, 2022

SUPERVISORS
Prof. Gerhard Lakemeyer, Ph.D.
Prof. Dr. Matthias Jarke

ADVISORS
Martin Liebenberg, M.Sc.
Tarik Viehmann, M.Sc.

Contents

1	Introduction	2
2	Related Work	4
2.1	Interoperability in Multi-Agent Systems	4
2.1.1	Semantic Interoperability using Ontologies	4
2.1.2	Semantic Interoperability using Agent Communication Languages	5
2.2	Service Discovery	6
2.3	Process Control	8
3	Structure of the Multi-Agent System	10
3.1	Provenance Information with the FactDAG Model	10
3.1.1	Theoretical Background	11
3.2	Ontologies and Semantic Information	11
3.2.1	Used Ontologies	12
3.3	Inter-Agent Communication with JSON-LD	14
3.4	Golog, ConGolog and IndiGolog	15
4	Implementation of the Multi-Agent System	18
4.1	Process Overview	18
4.2	Basic Agent Structure	20
4.2.1	FactDAG Implementation with factlib.js	21
4.2.2	Agent Service Cache and Service Discovery	22
4.2.3	Process Control Module	25
4.3	Specific Agent Services	26
4.4	User Interface	30
5	Discussion and Future Work	33
5.1	Service Quality and Efficiency of the Multi-Agent System	33
5.2	Process Control and Service Discovery	36
6	Conclusion	38
	Bibliography	39
	Appendices	43
A	Service Ontology	43
B	IBF Ontology	44
C	Sequence Diagram	45
D	Folder Structure and Files	46

1 Introduction

Already in 1998 A. W. Colombo stated that the manufacturing industry is and will continue in the future to be one of the main wealth generators for the world economy [Col98]. The importance of this sector caused the competition to grow rapidly. Since in the last decade the world moved towards a global economy with demanding products of higher quality and lower costs, new and existing production enterprises need to fulfill new requirements in terms of quality, response, agility and flexibility to be able to compete [Lei08]. This automatically requires companies to work together, forming a global network of production and manufacturing enterprises.

In this context, the goal of the research cluster Internet of Production (IoP) at the RWTH Aachen University is to create an infrastructure that enables data from different production domains to be available and used in real-time, which would therefore enable a new way of inter-domain cooperation. A concept to reach this goal is the idea of the World Wide Lab (WWL) agents. Those agents are attributed to different authorities which hold local and often dynamic information. The agents offer different services and work together to achieve a certain goal, for example the creation of a production plan for a certain product. In order to fastly create new information and use existing information from different domains, important components of the WWL are Digital Shadows and the FactDAG Model. Digital Shadows are compact or reduced views of physical processes or simulations [ML20], with which an agent can retrieve new information without creating congestion in the whole process. The FactDAG Model described by Gleim et al. [LG⁺20] is used to keep track of the provenance of information and enable data interoperability by structuring data as a directed acyclic graph of facts.

This thesis describes a system of WWL Agents based on an existing implementation, which was created and described by Nikola Balog in the context of the textile industry [Bal20], in order to address new problems like service discovery and process control in the context of the automotive industry. The specific use case of our extension is the creation of a production plan for a drive shaft, which is a component of vehicles that transfers the torque from the gearbox to the wheels. Depending on the vehicle it is used for, the drive shaft needs to be out of a material with specific properties.

This is a fitting problem for a multi-agent system (MAS), because the forging of a drive shaft is a complex process that consists of multiple steps located at different authorities. Specifically, the creation of a drive shaft partly consists of a **Forming**, a **Heat Treatment** and a **Machining** step, which are in our case correspondingly performed at the Institut für Bildsamer Formgebung (IBF), the Institut für Werkstoffanwendungen im Maschinenbau (IWM) and the Werkzeugmaschinenlabor (WZL) at RWTH Aachen University. A structural overview of the MAS can be seen in Figure 1.1.

In this thesis, we describe the theoretical components and the practical implementation of the WWL Agents, which are offering a service that provides the user with a production plan for a drive shaft. For that, we first look at other approaches for multi-agent systems in Chapter 2. Specifically, we look at interoperability, service discovery and process control. Then, in Chapter 3 we describe the structure and the theoretical background of

the components of our MAS, with the FactDAG Model, the different used ontologies, the concept of inter-agent communication and the programming languages Golog, ConGolog and IndiGolog. After that, in Chapter 4, we detail the actual implementation of the different components. We begin with the agent service cache, the process control module and the specific types of services, which are provided by the agents. At the end of the chapter, we describe the implementation of the user interface, which is created by a service broker agent. After describing the implementation, we discuss the quality of the system in Chapter 5 by examining the results of the services, the implemented approach to process control and service discovery, and by outlining the open challenges and future work. Finally, in Chapter 6 we conclude and summarize the results of this thesis.

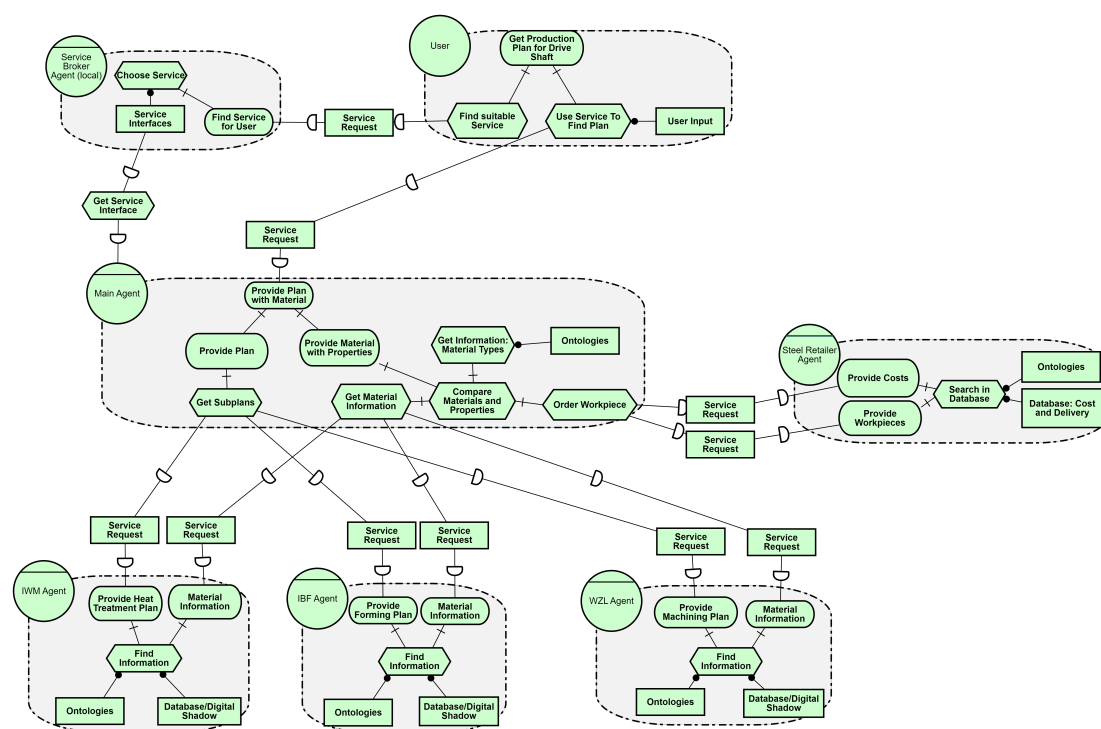


Figure 1.1: A model of the participating authorities attributing agents to the World Wide Lab¹.

¹The model was created according to the i-Star 2.0 model described by Dalpiaz et al. [FD16] and using the pi-Star editor: <https://www.cin.ufpe.br/~jhcp/pistar/>.

2 Related Work

In this chapter, we are going to look at the related work in some important fields for multi-agent systems in productional contexts. First, we will discuss semantic interoperability, which is important since we have different domains and authorities cooperating and therefore need a way to meaningfully interpret foreign data. Then we examine different approaches to service discovery. The WWL is a very dynamic network and thus we need a senseful way of discovering which agents, that are providing services, are part of the network. Lastly, we debate different techniques of process control to handle errors in the production process and in order to offer fitting alternatives of production plans.

2.1 Interoperability in Multi-Agent Systems

Schneider et al. specified three different ways of achieving of semantic interoperability: (semantic) interoperability **by chance**, **by standardization** and **by mapping** [MS⁺18]. Using interoperability by chance would mean that an interoperability framework allows any kind of information model and that those are only interoperable by chance. Using standardization means that all authorities agree upon a single standardized information model that all participants must use, which is the most common way of achieving semantic interoperability [AB17, SFV07]. Interoperability by mapping is described as the most powerful way of interoperability since it allows different information models. The problem is that the mappings between models need to be defined and that can only be done manually [MS⁺18, Lau05].

Two technical ways of how other multi-agent systems achieve semantic interoperability through standardization are firstly giving data semantical meaning using an ontology-based approach [Tom10] and secondly *Agent Communication Languages* (ACLs) [SV11]. We implement an ontology-based approach, but we will also discuss ACLs and how they differentiate from our approach. A chronological overview of ACLs and other standards for agent-oriented software engineering can be seen in Figure 2.1.

2.1.1 Semantic Interoperability using Ontologies

Nikola Balog, who developed the framework for WWL Agents this thesis is extending, describes multiple implementations using these two different approaches for semantic interoperability: *symbIoTe* [MS⁺18], *BIG IoT API* [AB17] and *DAML* [SAM01, Bal20]. The Framework symbIoTe requires all participants to implement the Interworking Interface and uses a Core Information Model (CIM) with Extensions (Platform Specific Information Model (PIM)), with the information models being standardized as RDF/OWL2-based ontologies. The BIG IoT API implements a standardized interface, which can be read and interpreted by the agents. It uses the Thing Description and JSON-LD, which we describe in Sections 3.2.1 and 3.3. Lastly, in the framework developed by McIlraith et al. [SAM01], agents written in ConGolog have an interface describing the services the agents offer. It enables a variety of agent technologies for automated web service

discovery, execution and composition. The service interface describes the input and output of the service, as well as information about interoperation and composition for more complex services so that an agent can automatically interact with a service.

In the implementation by Nikola Balog, on which our extension builds, several aspects from the approaches described above are combined: there exists an interface for each agent, describing the services an agent offers with the required input (the outputs are implicitly encoded in the service types). Those interfaces are written in JSON-LD and use the Thing Description like the BIG IoT API. Each agent uses the same RDF/OWL2-based core ontology, but it is also allowed for agents to implement different extensions of the same core ontology. In addition to that, we standardize the service result by providing meta-data describing the provenance of data used in the service execution and information about how the execution was adjusted, in order for it to be successful. Therefore, we create a standardized service output interface, which is similar to the approach by McIlraith et al. [SAM01].

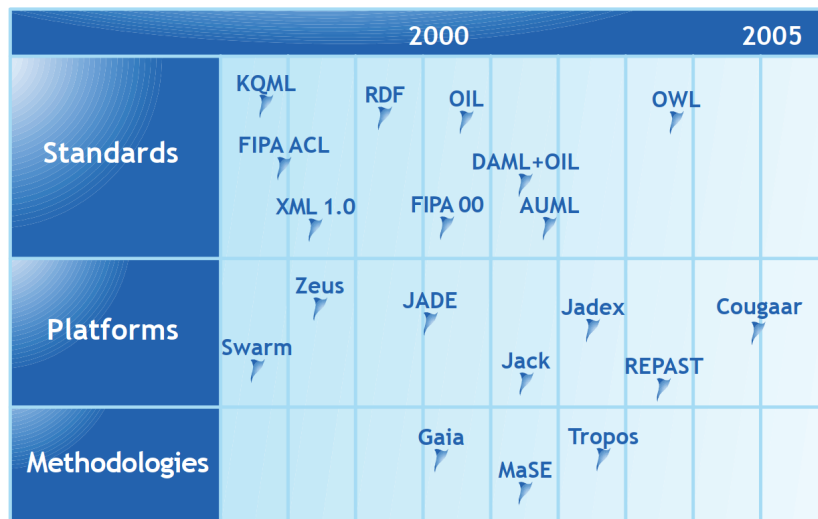


Figure 2.1: Some standards, platforms and methodologies of agent-oriented software engineering [LM06].

2.1.2 Semantic Interoperability using Agent Communication Languages

In 1992 Finin et al. described the *Knowledge Query and Manipulation Language (KQML)* as the first standard for agent communication languages (ACLs). ACLs are based on speech act theory, which views human natural language as actions like requests, suggestions, commitments and replies [SV11]. Speech acts are defined by the use of standard keywords, known as *performatives*. A message between two agents can then contain several parameters such as the sender, receiver, language and ontology of the content as well as the content itself. By adding these parameters the message and its content are given semantical meaning, depending on which performative is used [BCD02]. For example agent *A* could use the performative *ask-one* to ask Agent *B* for the address of a specific website. Agent *B* could then send the message or answer using the performative *tell* to indicate that he is now giving information, and not requesting it. The semantical information of a message itself, which is encoded in the performative when using

ACLs, is instead encoded in the Internationalized Resource Identifier (IRI) [DS05] of the corresponding service in the service ontology when using the ontology-based approach.

FIPA Reference Model and JADE The Foundation for Intelligent Physical Agents (FIPA) is an international organization that is dedicated to promoting the industry of intelligent agents by openly developing specifications supporting interoperability among agents and agent-based applications [FIP02]. The structure of the FIPA reference model, the normative framework within which FIPA agents exist and operate, consists of the following components: an agent exists on an *Agent Platform*, which provides an *Internal Platform Message Transport* to send and receive messages using the *Agent Communication Channel*. Messages can be directed at any other agent, whose location is saved in the optional *Directory Facilitator*. The mandatory *Agent Management System* exerts supervisory control over the use of the Agent Platform, which itself consists of the machines, operating system, agent support software, FIPA agent management components and all agents [FIP02, FB99].

The *Java Agent Development Framework (JADE)* is an in Java implemented software framework for the development of intelligent agents. It is implemented according to the FIPA-ACL standard [FB99] and therefore offers features like for example a FIPA-compliant Agent Platform [FB00].

Both KQML and JADE are based on the FIPA reference model of agent platforms. Our multi-agent system is not implemented according to this model, since we want to avoid centralization, so that the system does not depend on single nodes, but is robust against failures. Specifically, we do not have a Directory Facilitator or an Agent Management System, but rather distributed process control and dynamic and locally cached service discovery (see Section 4.2).

Using JADE would restrict a developer of an agent, that is going to be a part of the MAS, to using Java, where we want to allow multiple agents to have different architectures. For example our production plan agent is implemented using an IndiGolog implementation in Prolog (see Section 3.4) and the other agents are mostly programmed in Python.

2.2 Service Discovery

Since an agent in the MAS needs to cooperate with other agents in the network to achieve a goal or to provide a service, it needs to know its environment, i.e., all in the MAS available services. In order for one agent of the network to identify other agents with their services, we need *service discovery*. Del Val et al. [EdV09] describe different approaches to handle service discovery: those are **peer-to-peer**, using **coalitions** and using **middle agents**.

In peer-to-peer approaches agents broadcast a query across the whole network in order to identify themselves to the network or other agents in the network. The main drawback is the traffic overhead caused by the communication between all the participants in the network. Another way of service discovery that decreases this overhead is the use of coalitions or clusters, which are groups of agents that are somehow meaningfully combined. The communication then only happens between different coalitions, however forming senseful coalitions is a difficult task [EdV09]. The last approach is the distribution of middle agents or facilitators. Here the middle agents are agents with the distinct

function of matching agents, that are searching for a service, with the corresponding agent that offers the service. This centralized approach has the lowest communication overhead, however those middle agents cause a structural overhead and the whole system depends on them, which makes the network less robust against faults.

PDP The *Pervasive Discovery Protocol (PDP)* was developed by Campo et al. [CB02] to realize service discovery in pervasive multi-agent-systems. Pervasive systems are highly dynamic, with devices continually coming and going. Because centralized approaches, i.e., having a single device permanently present in the network, are not reliable in such systems [CB02], they use a peer-to-peer approach: a device announces its service only when other devices request the service. Those announcements are then broadcasted across the whole network, so that all other devices get to know of the new service without actively querying for it.

SSDP The *Simple Service Discovery Protocol (SSDP)* by Goland et al. [YYG99] was created as a discovery protocol for the Universal Plug and Play (UPnP) initiative and uses multicast-based discovery. It usually works with a central directory service, which is called the Service Directory (SD), but it can also work decentralized. When a new agent joins the network, it sends a multicast across the network or a unicast to the SD to notify its presence, where the SD records the location of the new agent. Other agents can then request a service by sending a unicast request to the SD.

SLP The *Service Location Protocol* [Vei97] is a standard for enabling IP networks-based applications to automatically discover the location of a required service. They use three types of agents: *User Agents*, which perform service discovery on behalf of users or clients, *Service Agents*, which advertise the location and attributes of services, and *Directory Agents*, which store information about the services announced in the network [Cam02]. User Agents can send unicasts to Directory Agents, which are actively crawling service information, to discover services in the network.

Jini *Jini* [Wal99], now called *Apache River*, is a framework by the Apache Software Foundation for the implementation of distributed software applications. In the framework, service discovery is handled by a directory service called the *Jini Lookup Service (JLS)*. Other agents have to lease a registration for their services at the JLS and, if they want their service to stay available, renew the registration regularly. Otherwise, the lease expires and the service registration is dropped [Arn99].

OWLS-MX *OWLS-MX* [KFS06] is a matchmaker for services and requests specified in OWL-S [DM⁺07], which is semantic markup for web services. In the implementation by Klusch et al., a specific middle agent or matchmaker agent processes incoming requests by applying the described algorithm. The algorithm combines logic-based reasoning and content-based (e.g., on the service description) syntactic similarity measurements.

For our WWL Agents we implemented a peer-to-peer approach (see Section 4.2.2), because we want to prevent high dependencies on single network authorities. Each service has its own initially empty service cache, where it can save the location and description of a service that is offered by some agent in the network — similar to the SD in the SSDP, but decentralized. If the cache does not contain an address for a

required service, an agent performs an on-demand multicast (similar to the PDP) to ask all participants of the network for their location. The multicasting agent can then use the location of the agents to retrieve information about the services that every agent offers. Then, the agent caches the information of the required service and only needs to directly communicate with the corresponding agent, if it needs to request the same service again later. This approach reduces the communication overhead and is robust against failures of single agents, because it is decentralized. However, it has a storage overhead since each agent caches the service information of eventually all other services. This drawback can be afforded since the needed storage is equally distributed across the network and the robustness against failures is in our context of higher importance to us compared to the storage overhead.

2.3 Process Control

In a MAS, different agents depend on each other in order to achieve a superior goal. If one of the agent fails, one needs process control to handle errors and offer alternatives to the user. In 1991 Diltis et al. [DD91] describe four different structures for process control: **centralized**, **hierarchical**, **modified hierarchical** and **heterarchical** process control.

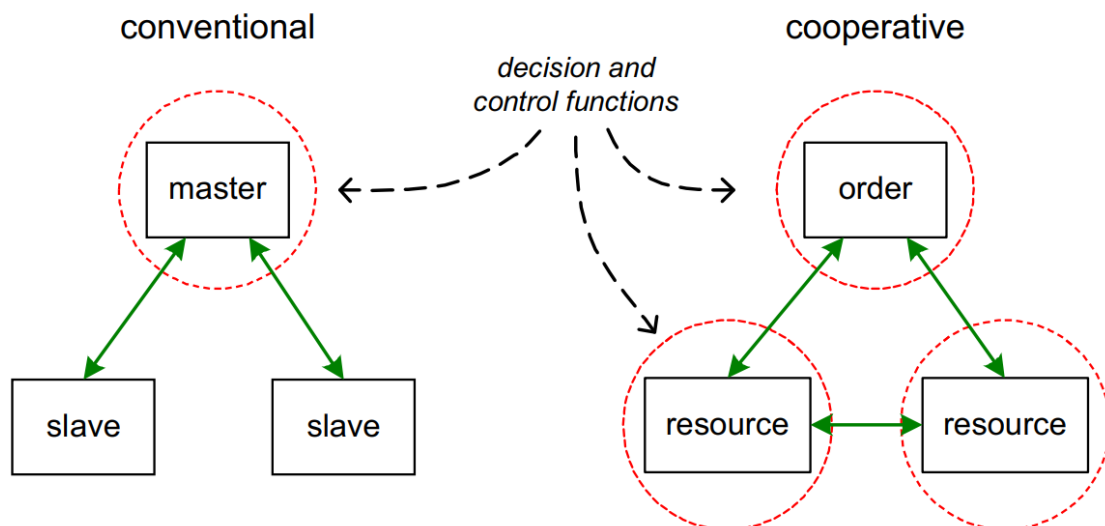


Figure 2.2: Conventional and centralized process control versus cooperative and distributed process control [Lei08].

In the centralized approach there exists a single node or agent, where all planning and processing information are concentrated. This architecture is easy to optimize, but has some disadvantages in terms of response time, tolerance to faults and scalability. In the hierarchical model there are several control levels, allowing the distribution of decision-making to multiple layers, as seen on the left in Figure 2.2. This makes the system more robust, predictable and efficient, however disturbances in the system still reduce the performance significantly. The modified hierarchical system tries to reduce those disturbances by additionally adding connections between nodes of a layer. In the heterarchical structure, decision-making is handled locally and autonomously without a global view of the system, which is a very robust structure against failures of single nodes [Lei08].

ADACOR The ADACOR holonic control architecture described by Leitão et al. [PL06] in 2006 is a decentralized manufacturing control system. In ADACOR, complex problems are divided into several small problems, each solved by a single control unit. Each control unit has its own objectives and knowledge, but no global view of the whole system. In order to reach a production decision, the single units, which should be re-configurable, robust, reusable and able to learn, have to work together. Additionally, the control units are often connected to real physical automation devices like sensors.

MASBO The *Multi-Agent System for Building Control* (MASBO) [QLG06] is a system for building automation and management, that enables energy-efficient and occupants satisfied building control. There are three general types of agents: *personal agents* (assistants that manage user specific information), *local agents* (mediators and information providers) and *central agents*. The central agents are mainly responsible for decision aggregation, i.e., gathering sub-decisions from local agents and then making a final decision.

Hierarchical MAS Wakulicz-Deja et al. [WD⁺09] describe a hierarchical multi-agent system for process control. They focus on the problem of making a global decision based on the decision rules of local expert agents. Every agent is called a *Resource agent* because it has access to some data and resources. Additionally, agents that establish a decision value for an issue are called *Synthesis agents*, which receive their tasks from a central *Manager agent*, which has a global overview of the system.

We implemented a distributed approach for process control (see Section 4.2.3) very similar to the ADACOR architecture. Each agent has an (optional) process control module, that handles locally detected errors and communicates them with a solution, if possible, appropriately across the network. Having a conventional centralized process control system would work against our intention of making the MAS decentralized, in order to avoid dependencies on single network nodes. Distributing process control across the MAS also distributes the computation load, since more complex problems can be divided into smaller, dispersed issues.

3 Structure of the Multi-Agent System

In this chapter, we are specifying the used technologies of the multi-agent system (MAS), which was implemented by Nikola Balog [Bal20] and which is expanded by this thesis. Since keeping track of the provenance of data enables the user to evaluate its quality, the FactDAG Model described by Gleim et al. was implemented by using the Typescript-Library factlib.js by Liam Tirpitz. After that, we give a quick summary of ontologies in general and describe the important ontologies that are used in the MAS. Lastly, we explain the context of IndiGolog and Golog, which are programming languages based on first-order logic, in which our production plan agent is written in.

3.1 Provenance Information with the FactDAG Model

The MAS combines data from multiple production domains and distributes it across the network. This results in multiple issues: Which authority created this data? Is it up-to-date or already deprecated? What quality does the data have?

In the context of the *Internet of Production* Gleim et al. derived five key requirements, based on the *FAIR* data principles described by Wilkinson et al. [MDW⁺16], which need to be addressed in order to solve these problems and to achieve large-scale data-interoperability for industrial processes. They also propose a model inspired by successful distributed version-control and collaboration systems which builds upon the *FactDAG Model*.

Findability F1 (meta)data are assigned a globally unique and eternally persistent identifier. F2 data are described with rich metadata. F3 (meta)data are registered or indexed in a searchable resource. F4 metadata specify the data identifier.	Accessibility A1 (meta)data are retrievable by their identifier using a standardized communications protocol. A1.1 the protocol is open, free, and universally implementable. A1.2 the protocol allows for an authentication and authorization procedure, where necessary. A2 metadata are accessible, even when the data are no longer available.
Interoperability I1 (meta)data use a formal, accessible, shared, and broadly applicable language for knowledge representation. I2 (meta)data use vocabularies that follow FAIR principles. I3 (meta)data include qualified references to other (meta)data.	Reusability R1 meta(data) have a plurality of accurate and relevant attributes. R1.1 (meta)data are released with a clear and accessible data usage license. R1.2 (meta)data are associated with their provenance. R1.3 (meta)data meet domain-relevant community standards.

Figure 3.1: A summary of the FAIR data principles [LG⁺20].

3.1.1 Theoretical Background

The *FactDAG Model* described by Gleim et al. is a data layer model, that maintains the provenance information of data by structuring data as *Facts* in a directed acyclic graph [LG⁺20]. It addresses the following five issues, which are based on the four principles shown in Figure 3.1:

1. **Immutability:** Data must be immutable, i.e., it can not be overwritten or changed.
Referenceability: Data must be referenceable by some kind of identifier.
Unique Identifiability: The identifier has to be unique for the data across the whole system.
2. **Authorative Source:** Data must be clearly attributed to a responsible authority, which is a singular source of data.
3. **Versioning:** Data must be versioned, to order data revisions and indicate whether the data is up-to-date or deprecated.
4. **Data Sovereignty:** Data must be subject to definite legislation and usage regulations.
5. **Stakeholder Confidentiality and Minimalism:** Data access should only be granted to the absolute minimum of required information. Data-sharing should occur in a privacy-respecting manner whenever possible.

The requirements 1-3 are met by lifting data to immutable **facts**. Data is internally, that means by the authority *auth* that owns the data, referenced by an internal and unique unifier — the *id*. Additionally, a revision identifier *rev* indicates the current version of the fact. In total the triple $\langle auth, id, rev \rangle$ is referred to as the **FactID** and serves as a persistent identifier (PID), by which the data can be referenced globally.

The WWL Agents use this model partially by also lifting data to facts and therefore fulfilling the requirements 1-3 but not requirements 4 and 5, which need to be addressed in future work. Specifically, *factlib.js*, which is a library for Node.js¹ developed by Liam Tirpitz [Tir20], is used for the FactDAG Provider. The specific implementation and usage of *factlib.js* will be further described in Section 4.2.1.

3.2 Ontologies and Semantic Information

Ontologies are used to formally model the structure of a system and its entities, which are abstract representations of physical or virtual objects [Con04]. Using ontologies for structuring and standardizing data enables interoperability and the inference of implicit knowledge [XW⁺04].

In our use case, which is the creation of the production plan for a drive shaft, we use ontologies for the materials, the material properties and the services, which are offered by the WWL Agents in order to achieve semantic interoperability. The creation process consists of different substeps, being a forging, heat treatment and machining subprocess, which are handled by different agents. These agents are, correspondingly to the described substeps, the **IBF Agent**, the **IWM Agent** and the **WZL Agent**,

¹<https://nodejs.org/en/>

which all hold information about the subplans and material properties. In addition to that, data about other material properties, the costs and workpieces, which have to be ordered, are managed by the **Steel Retailer Agent**. The **Production Plan Agent** finally creates the production plan using the services and data of the other agents. In the following section we describe the specific ontologies, which are used by these agents to give shared data global semantical meaning.

3.2.1 Used Ontologies

The MAS uses several ontologies to specify semantics of data as well as enable inter-agent communication. In this section we describe the **IBF** ontology specified for our use-case, the **P-Plan**² ontology, which is used to describe industrial production plans and processes, the **Thing Description**³ ontology, which we use for the service descriptions of the agents, as well as the **Service** ontology, which describes the types of services and the metadata of a service result.

IBF Ontology The Institut für Bildsame Formgebung (IBF) provides an ontology that encodes information about materials, processes and workpieces that are required for the creation of a drive shaft. Figure 3.2 gives an overview of the ontology. It shows the three general process types *Forming*, *Heat Treatment* and *Machining* under *Process*. A *Property* is an entity that describes another entity, for example the *Material Properties* are the properties of a material, such as the *Weldability* and the *Yield Strength*. The materials that are considered for the drive shaft are listed under *Material*. One example for a suitable material is *Acidur4301*, which is a type of *Steel*.

Later this ontology can be extended to further be able to represent the machining, heat treatment and forming processes more precisely, for example using the P-Plan ontology which is described in the next paragraph.

P-Plan P-Plan is an ontology that describes how industrial plans are composed and their correspondence to provenance records that describe the execution itself [DG14]. A **p-plan:Step** is part of a **p-plan:Plan** and may be preceded by another step. Each step has input and output variables, which all are **p-plan:Variables**.

Therefore a plan is an ordered list of process steps, which use some input entities and perform processes that result in a change in the environment or other output entities.

We can use the P-Plan ontology to formalize production processes (in our case the complete forging process of a drive shaft), while parallelly maintaining the provenance information of actions and entities that are parts of the process (in our case for example the information that the IBF Agent provided the forming subplan). Formalizing the production plans with an ontology has the additional advantage, that we can later automatically visualize it as a network diagram.

Thing Description The Thing Description ontology is a formal model and representation of a Web of Thing (WoT) Thing Description [Con20]. This Thing Description consists of the metadata and interfaces of a *Thing*, which is an abstraction of a virtual or physical entity that is somehow part of the Web of Things. We use the Thing Description to standardize our agent interfaces. Most often Thing Descriptions are encoded in

²<https://www.opmw.org/model/p-plan/>

³<https://www.w3.org/TR/wot-thing-description/>

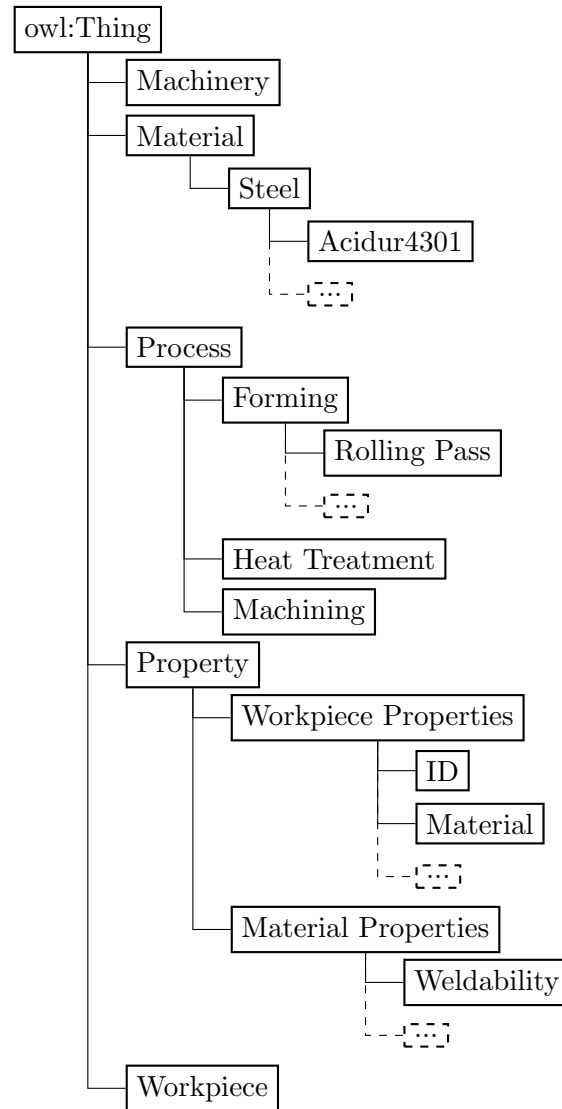


Figure 3.2: Overview of important parts of the ontology provided by the IBF. An extended view can be found in the appendix.

a JSON format and allow JSON-LD processing, as we will further describe in the next section.

Service Ontology The service ontology represents which types of services exist in the MAS, as well as which (optional) metadata a service result contains. The service type defines what a service provides, i.e., what the service result is going to be. For example, whether it is a list of material properties or a plan of the forming subprocess. In Figure 3.3, which gives an overview of the service ontology, one can see that the ontology describes entities called *Service Parameters*. Service parameters are additional pieces of information that describe the execution of a service. These are for example the authority that offers the service, the authorities, whose services are requested during the initial service execution and optional information about how the input got altered, in order for the service to be executed successfully.

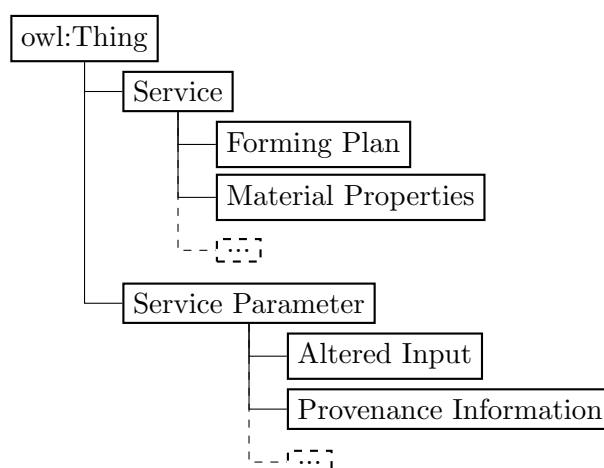


Figure 3.3: Overview of important parts of the service ontology. An extended view can be found in the appendix.

3.3 Inter-Agent Communication with JSON-LD

In our MAS, we use the Thing Description presented in Section 3.2.1 to describe the services that our agents offer. The agent interfaces are written in **JSON-LD**⁴. The JSON Linked Data format is easily human-readable and writable and based on the JSON format. It makes data machine-readable by representing data in terms of ontologies, which are specified with the **context** keyword.

As shown in Listing 3.1, under **td:actions** all the actions the agent can perform are listed. Each action has a **td:title**, a human-readable **td:description** as well as **td:forms**, which describe how the action or service can be accessed. The **td:input** is a list of all the inputs that the agent needs in order to successfully complete the action. As seen in our example, the IBF Agent offers two different services, one being the *Material Properties* service which requires a specific material as input.

With every agent providing such an interface, other agents can automatically identify which types of service an agent offers, fill out the interface with semantically fitting information, successfully request the service and use the in the service result provided provenance information to update their local FactDAG.

```

1  {
2    "@context":
3    {
4      "td": "http://www.w3.org/ns/td/v1#",
5      "dss": "http://www.semanticweb.org/max/ontologies/2021/9/
        Services_Drive_Shift#",
6      "ibf": "http://www.semanticweb.org/mohamadalomari/ontologies
        /2021/6/IBF1107#"
7    },
8
9    "td:actions":
10   {
11     "dss:Material_Properties":

```

⁴<https://json-ld.org>

```

12      {
13        "td:title": "Material Properties",
14        "td:description": "This service provides information
15          about the properties of a specific material.",
16        "td:input":
17          {
18            "ibf:Material" : ""
19          },
20        "td:forms": [{"td:href": "http://localhost:10000/
21          material_properties"}]
22      },
23      "dss:Forming_Plan":
24      {
25        "td:title": "Forming Plan",

```

Listing 3.1: The agent interface of the IBF Agent written in JSON-LD. The agent, which is described here, offers services to provide material properties or a forming plan.

3.4 Golog, ConGolog and IndiGolog

The situation calculus [McC63, Rei01] is a logic formalism, where actions are described with their corresponding effect on the modeled world. In the situation calculus, one defines actions, the situations when the actions are *possible* and what *effect* an action has, i.e., the successor state or situation after executing an action. For example [GDG09], let the action *pickup* object x be possible, when nobody is holding x and x is not heavy. Then this is expressed in the situation calculus as

$$Poss(pickup(x), s) \equiv [(\forall z) \neg holding(z, s)] \wedge \neg heavy(x)$$

where s is the current situation. A predicate or function whose argument is a situation and whose value can change over time is called a *fluent*. In our example the expressions

$$is_carrying(robot, x, s)$$

$$\text{and } location(robot, s)$$

are fluents (the first being relational and the latter functional), because their values depend on the situation s . In contrast to that, the relation *heavy* is independent of the situation s , and therefore is not a fluent. In addition to the precondition, under which the action can be performed, world dynamics are specified by the effects of an action (effect axioms). For example the effect of *dropping* a *fragile* object can be defined as following:

$$Poss(drop(r, x), s) \wedge fragile(x, s) \supset broken(x, do(drop(r, x), s))$$

I.e., if an object can be dropped and is fragile, the effect of dropping the object is that the object is broken in the successor state s' . The next state s' can be expressed using the binary function $do(a, s)$, which denotes the successor state s' of executing the action a in the state s .

Golog [HJL97] is a programming language based on first-order logic. Its interpreter automatically maintains an explicit representation of the dynamic world that is digitally modeled and is based on the situation calculus. Golog is a well-suited instrument to describe the behaviour of robots and agents, since it allows a developer to directly define high-level instructions. **ConGolog** (Concurrent Golog) [GDG09] is an extension of Golog that was later developed to allow concurrent programming for more complex agent behaviour. In Golog concurrency was not possible, ConGolog however allows parallel actions like interrupts and (prioritized) concurrent processes. Finally **IndiGolog** (Incremental Deterministic Golog) [GDG09] enabled the programmer to control planning, use online-execution, implement sensing the environment and monitor the execution. We are using an IndiGolog interpreter for the implementation of our more complex agents. The IndiGolog interpreter itself is implemented in **Prolog** [WSTL12]. It offers the following constructs from the situation calculus:

- **prim_action**: `prim_action(action)` declares that a primitive action named *action* exists.
- **poss**: `poss(action, condition)` declares that the action *action* is executable when the condition *condition* is valid.
- **execute**: `execute(action, result)` declares how the action *action* is to be executed and that the result should be *result*.
- **proc**: `proc(proc, actions)` declares that there exists a procedure called *proc* which is a construct of multiple other actions.
- **prim_fluent**: `prim_fluent(fluent)` declares that there exists a fluent called *fluent*.
 - **initially**(*fluent*, *value*) declares the initial value of the fluent.
 - **causes_val**(*action*, *fluent*, *value*, *condition*) describes the effect of an action on the fluent when the condition is valid.

An exemplary program is shown in Listing 3.2. Two actions, *reset* and *swap* are defined using **prim_action**. Both actions can be executed, when the value defined in the predicate **poss** is true, which is here always the case. Then, with **prim_fluent** two fluents *x* and *y* are defined, which have the initial values 0 and 100. The predicate **causes_val** is now used to specify how the value of a fluent changes, when a specific action is called. One can see that the action *swap* increases the value of *x* by one and correspondingly decreases the value of *y* by one. The other action *reset* resets the fluents to their initial values. In addition to how an action changes the value of a fluent, one can describe the execution body of an action using the predicate **execute**. The execution body consists of pure Prolog code, that gets executed when the corresponding action is called. In addition to the constructs known from the situation calculus, IndiGolog provides high-level concepts known from imperative programming languages:

- **A, B** - sequence of actions *A* and *B*.
- **if(cond, A, B)** - if the value of *cond* is true, execute *A*, otherwise *B*.
- **while(A, condition)** - execute action *A* while the condition is valid.
- **conc(A, B)** - execute the actions *A* and *B* concurrently.

- **pconc(A, B)** - concurrent execution where *A* has a higher priority than *B*.

These concepts from imperative programming languages can now be used when defining more complex procedures consisting of other procedures and primitive actions. In the example the predicate **proc** is used to define the procedure *control*. When *control* is called, the action *swap* gets executed until $y = 0$, then the primitive action *reset* is performed.

```

1  prim_action(reset).
2  poss(reset, true).
3  prim_action(swap).
4  poss(swap, true).
5
6  prim_fluent(x).
7  initially(x, 0).
8
9  prim_fluent(y).
10 initially(y, 100).
11
12 causes_val(reset, x, 0, true).
13 causes_val(reset, y, 100, true).
14 causes_val(swap, x, x+1, true).
15 causes_val(swap, y, y-1, true).
16
17 execute(_,_) :- true.
18
19 proc(control, [
20     while(y>0, swap),
21     reset
22 ]).

```

Listing 3.2: Example code for a program written in an IndiGolog implementation in Prolog. The procedure *control* consists of a loop, which decreases the value of the fluent *x* and increases the value of the fluent *y* by one, until $y = 0$, where it resets *x* and *y* to their initial values.

4 Implementation of the Multi-Agent System

In this chapter, we describe the specific implementation¹ of the multi-agent system, which is responsible for creating a drive shaft production plan based on user input parameters. For that, we are first going to examine the distribution of knowledge in our use case and then give an overview of the process. From this process structure we derive the types of services that are offered in the MAS and which are needed in order for the production plan agent to create a production plan. After that, we look at the general basic agent structure, i.e., the components that every agent implements. These are:

- a **FactDAG Storage** with **factlib.js**,
- an **Agent Service Cache**,
- and a **Process Control Module**.

Finally, we describe the functionality of the Service Broker Agent, which is responsible for providing the user with a clear user interface. This interface enables the user to access all services of the system by searching for them using a google-like search.

4.1 Process Overview

As we have already mentioned, the production of a drive shaft is a complex process that consists of multiple steps located at different authorities. The substeps, that we consider, are

- a **Forming** step located at the Institut für Bildsame Formgebung (**IBF**),
- a **Heat Treatment** step located at the Institut für Werkstoffanwendungen im Maschinenbau (**IWM**),
- and a **Machining** step located at the Werkzeugmaschinenlabor (**WZL**).

Before a workpiece can be processed, one first needs to choose a material. Depending on the use case of the drive shaft, different types of materials have different suitability. For that, we are going to quantify the suitability of a material by examining some of the properties a material has: the **Weldability**, the **Yield Strength**, the **Machinability**, the **Heat Treatability** and the **Costs per Kilogram**. The first four material properties are minimal and hard requirements for a material, whereas the user-specified costs are maximal and soft requirements.

Every authority (IBF, IWM and WZL) is represented by a single agent, where we have

¹The repository of the implementation can be found under: https://git-ce.rwth-aachen.de/Martin.Liebenberg/wwl-agents/-/tree/drive_shaft_agents.

an additional **Steel Retailer** Agent that provides information about costs and workpieces that can be ordered. All agents contain in total all the information needed for creating a complete production plan, but the data is distributed across the agents. Similar to the Steel Retailer Agent holding information about the costs, the IBF Agent holds information about the Weldability and Yield Strength of a material, as well as information about the forming subprocess. The other agents hold the same type of information about their corresponding expert areas.

The **production plan agent** can then use this data, which is accessible through services, that the agents, which hold the information, offer, to create the production plan. Initially, it owns no data, and is therefore only a functional agent. A structural overview of the complete system can be seen in Figure 1.1.

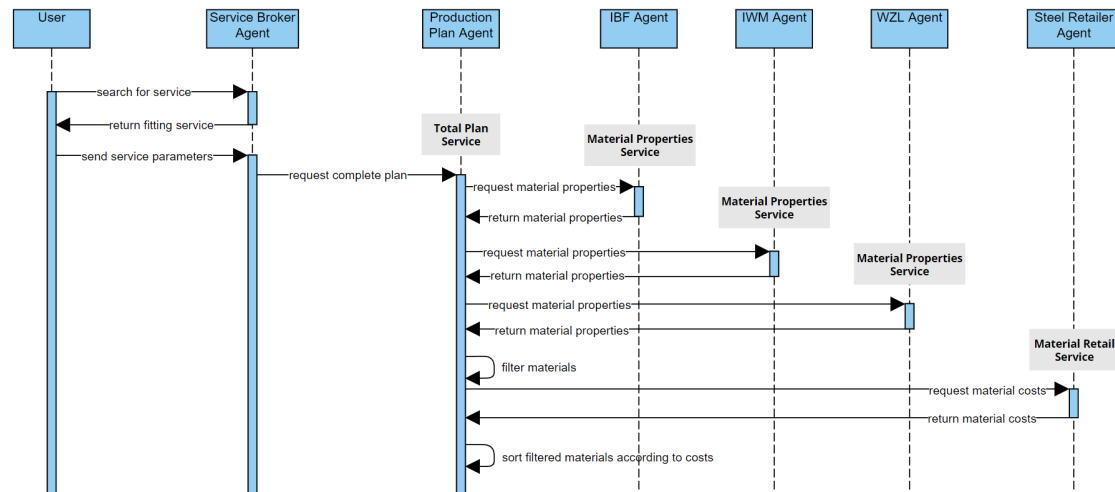


Figure 4.1: A sequence diagram of a user requesting the *Total Plan* service and the (inter)actions performed by the production plan agent and the other agents, when choosing the right material. The grey boxes indicate the service context of the performed actions.

Process of Production Plan Agent Service When a user requests the Total Plan service of the production plan agent, he specifies the mentioned minimal and maximal material properties. The production plan agent uses the IBF Ontology, to identify all of the possible materials types, that exist. It then requests the *Material Properties* services of the other agents, to gather data about the properties of all materials. After that, the materials, which do not meet the user set minimal requirements, are filtered. Then the production plan agent requests information about the costs of all the remaining materials by requesting the *Material Retail* service and chooses the cheapest material from them. The agent interactions when choosing the right material are visualized in the sequence diagram in Figure 4.1.

If no material meets the minimal user parameters, the production plan agent can not create a plan, but returns a textual answer, informing the user that no material with the minimal properties could be found. If a valid material can be found, but the costs are higher than the maximum limit, then the production plan agent creates a production plan but gives an additional warning, that the costs are above the limit.

After finding the right material for the drive shaft, the production plan agent orders a specific workpiece from the Steel Retailer by using the *Material Buy Workpiece* service.

Now it can request the subplans by using the corresponding services, which require a workpiece and a material as an input. All the services return, in addition to the result, information on the provenance of the service data. The production plan agent creates the complete production plan with the combined provenance information and returns it to the Service Broker Agent, which initially provided the user with the *Total Plan* service and now visualizes the resulting plan as a directed graph. The process of ordering the workpiece and creating the plan is shown in the sequence diagram in Figure 4.2.

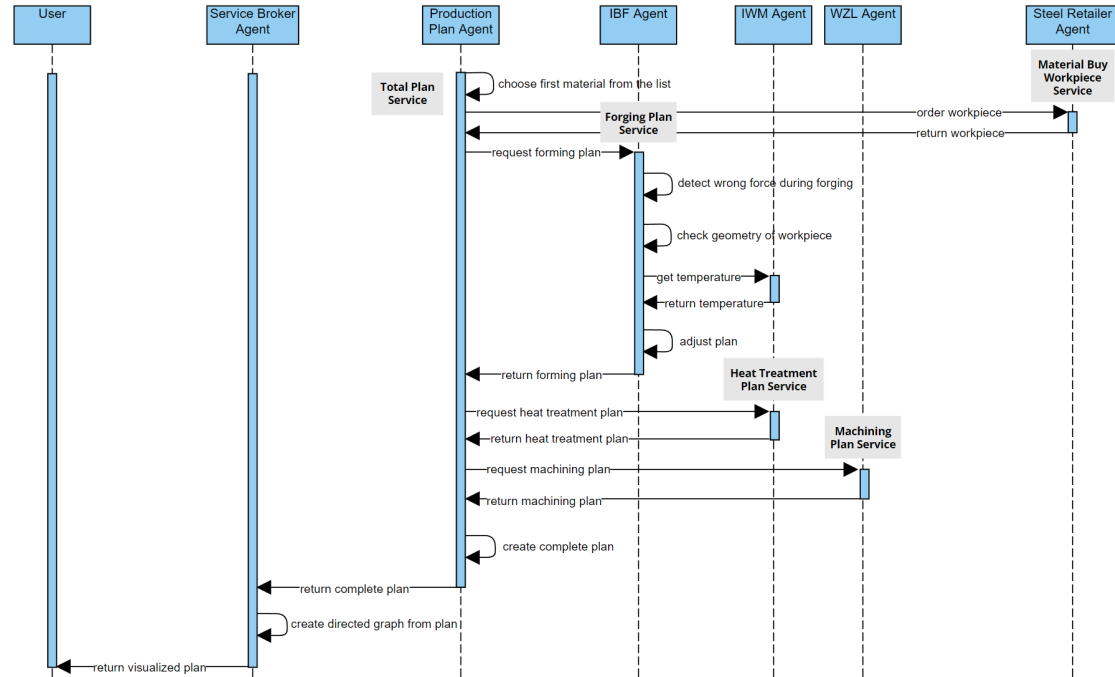


Figure 4.2: Continuation of the sequence diagram showing a user requesting the *Total Plan* service and the (inter)actions performed by the production plan agent and the other agents. Here displaying the process of requesting and combining the subplans, after choosing the best material. The IBF Agent adjusts the forming plan, because it detected an unexpected force during the forging step.

4.2 Basic Agent Structure

All of the in the MAS involved agents are structured as seen in Figure 4.3. A WWL Agent has a local FactDAG provider to store and modify data as facts, a service discovery module to identify services that are available in the MAS, a service interface that describes the own services, a process control module to handle failures and problems as well as an agent logic, that manages all the other components. Since the basic functionalities are the same for each agent, we implemented a single, globally used service discovery module and a script, which provides functions for the agents to managing their FactDAG storages. Since the process control is local and individual, the implementation differs for every agent, with some agents having no specific process control module, since their services are currently only providing static data. An overview of the folder structure and files that we are describing can be seen in Appendix D.

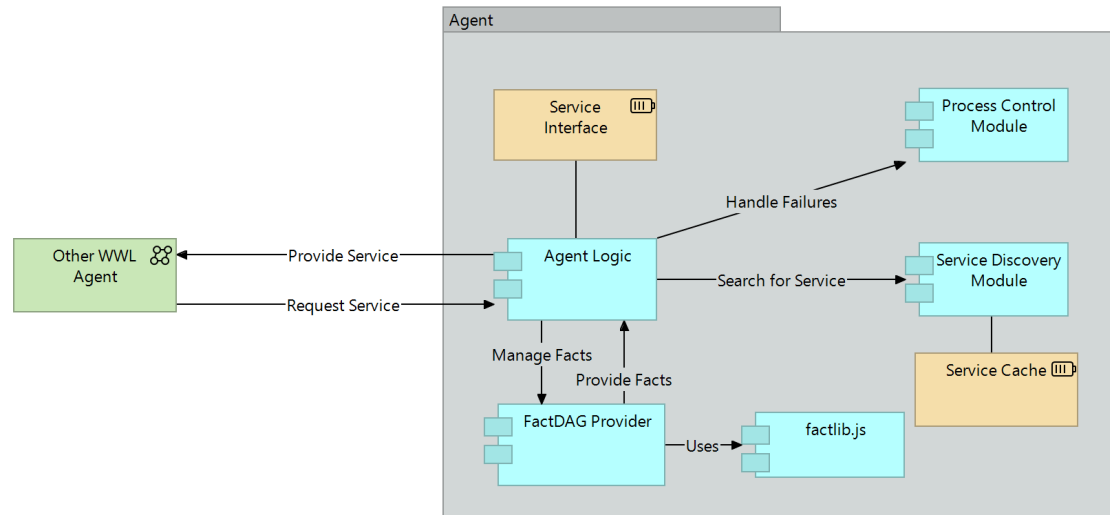


Figure 4.3: Overview of the basic agent structure. Every agent has some kind of *Agent Logic*, which uses a *FactDAG Provider* to store and administer data. The *Process Control Module* is responsible to handle locally detected errors. In order to access other services, the agent uses its *Service Discovery Module*, which discovers other agents in the system, requests the *Service Interfaces* and caches the information about how to request the other services.

4.2.1 FactDAG Implementation with factlib.js

The Typescript library *factlib.js*, which was developed by Liam Tirpitz [Tir20], provides functionality to structure and store data according to the FactDAG model. For managing the FactDAG providers of the agents we provide two scripts: The first script (an overview of the folders and files can be found in Appendix D), *setup.ts*, sets up the basic environment for the Trellis LDP² with the message broker and the fact and process directories for all the five agents. Therefore, it is only used to initialize the storage providers, when the system is newly installed. The other script, *edit_facts.ts*, provides functions for agents to create, delete and retrieve facts in their local data storage. The corresponding *edit_facts.py* file provides wrapping functions to call the corresponding functions from the typescript file. That way, the agents, whose logic is programmed in Python, can import the most important functionalities of *factlib.js* directly. The specific functions that are used by all agents to store their data in the local FactDAG storage are the following:

- **create_fact(authorityID, localFactId, mimeType, configPath, dataInPath)**
Creates a fact in the local FactDAG with the *authorityID*, referenced locally with a *localFactId*, containing the data stored at *dataInPath*, which has the type *mimeType*. For this, a config file located at *configPath* is required in *factlib.js*, which specifies the process and activity associated with the newly created fact.
- **get_fact(authorityID, localFactId, mimeType, configPath, dataOutPath)**
Retrieves the data of a local fact and stores it in a file located at *dataOutPath*. The other parameters have the same functionality as in **create_fact**.

²<https://github.com/trellis-ldp/trellis>

- **change_config(process_location, activity_location, configPath)**
Changes the process and activity of a config file located at *configPath*.

The general structure of how provenance metadata is added to the stored information can be seen in Figure 4.4. Data is stored as a *Fact* and attributed to an *Organization* or *Authority* (for example the IBF). The *Fact* was generated by an *Activity*, which is associated with an *Agent* (for example the IBF Agent).

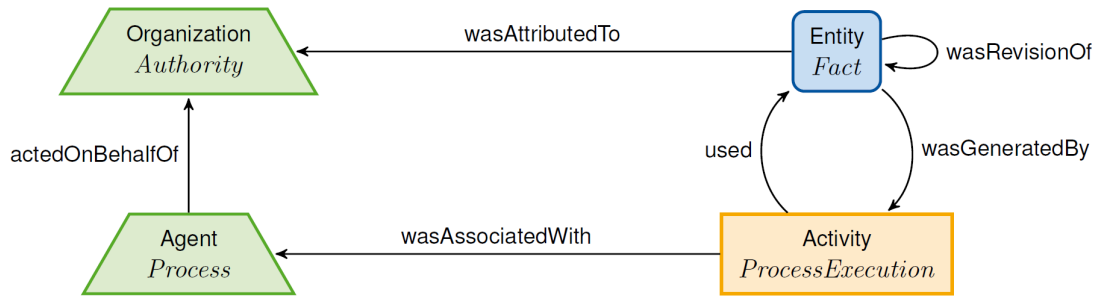


Figure 4.4: Relation between data and its provenance in factlib.js [Tir20].

The activities, which generate the facts, are in our case the service types, which provide the services. For example the fact, that stores the machining plan, is generated by **provideMachiningPlan**, which is the activity (or FactDAG Process) corresponding to the service type **Machining Plan**. Which and when other facts are created or retrieved from the FactDAG storages is further detailed in Section 4.3, when we describe the specific service types.

4.2.2 Agent Service Cache and Service Discovery

In the agent service cache, an agent caches information about the services that are offered by other agents in the system. For that, the agents need a standardized way of communication, similar to the Agent Communication Channel in the FIPA Reference Model [FIP02]. We handle the communication during the service discovery and the actual access and use of services on different communication layers. For the discovery process, we implement a **UDP** based multicasting for every agent, whereas the services of an agent can be accessed via **HTTP** requests.

UDP Multicasting for Agent Discovery The *global* module provides a script for handling UDP multicasting. An agent binds its own address to a unicast socket and a for the whole system static address to a multicast socket. Then, it can send messages containing its address through the multicasting channel, in order to identify itself to the other agents. When another agent receives the request message, it sends a direct answer containing the agent's address via the unicast channel. The agent, which requested the addresses of the other agents, can then request the service interface using HTTP requests and cache the information locally. An overview of the identification process can be seen in Figure 4.5. All agents execute the agent discovery process using the functions that are provided by the *udp.py* script:

- **udp_start_receiver(address, port, answer_callback)**
This function binds both a multicast and a unicast socket to a specific *address* and

port. The function starts a thread, which listens for messages, for every socket using the *listen_on_socket* function.

- **listen_on_socket(socket, address, port, answer_callback)**

This function listens for messages on a given *socket* with an *address* and *port*. The *answer_callback* gets executed, whenever another agent sends an answer to the local agent. In our implementation, the *answer_callback* is a function from the *cache_controller.py*, which requests the service interface of the agent via HTTP and then caches the information in a JSON file.

- **udp_request_addresses(address, port)**

Agents can use this function to request the service information of the other agents. When called, the agent sends a message in the multicasting channel containing its *address* and *port*. Other agents receiving the request message send an answer message via the agents unicast channel, containing their own addresses. The local agent can then request the service interface via an HTTP request and cache the contained information.

- **udp_reply_address(message, address, port)**

This function is used to send an answer to an agent, which requested the addresses of the other agent, i.e., which called the *udp_request_addresses* function.

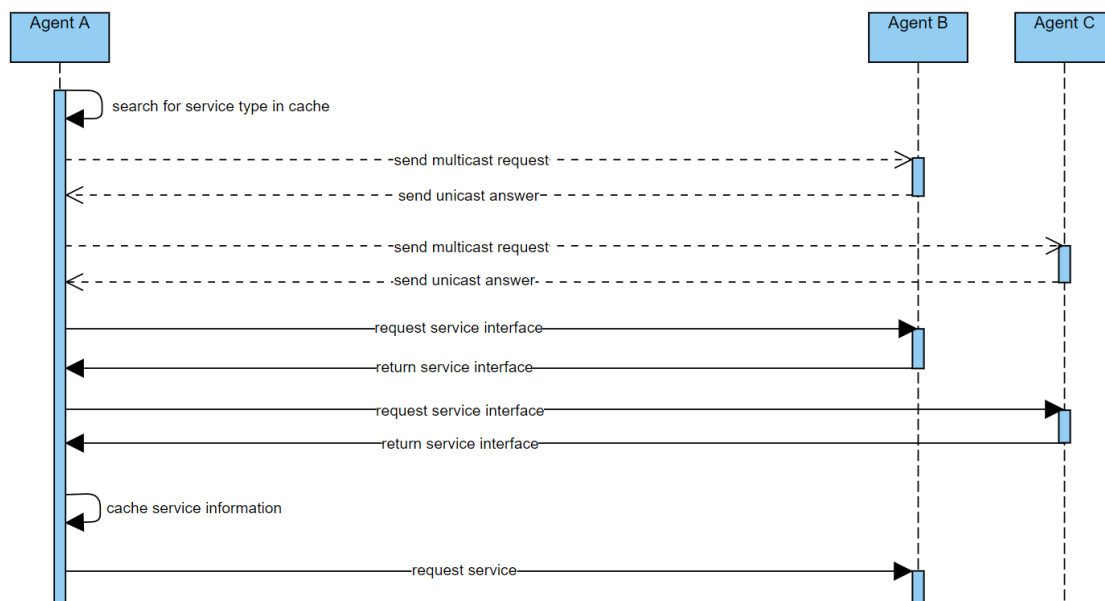


Figure 4.5: Example of agents executing the service discovery. *Agent A* needs to request a specific service type, but can not find an agent that offers a fitting service in his cache. It then sends a request message in the multicast channel, which is received by all agents in the system (here *Agent B* and *Agent C*). Those agents send an answer, so that the local agent can request the service interfaces and cache the service information. Finally, *Agent A* sends a service request to *Agent B*, which offers the service it searched for. Dashed lines indicate the use of the UDP layer, against which solid lines indicate HTTP requests as the way of communication.

Service Information Caching After the described identification process, an agent knows the addresses of the other agents in the World Wide Lab. It can now request the service interfaces, in order to get information about the offered services of another agent, and then cache it in the initially empty service cache. In Listing 4.1 one can see parts of the service cache of the Service Broker Agent. For every agent (identified with their address), it caches the services with a human-readable **title** and service **description**, the **IRI** of the service type, the **route**, which is the address, that can be used to access the service, and the **costs** of accessing the service.

IRI stands for *Internationalized Resource Identifiers* and standardizes the concept of *Uniform Resource Identifiers* (URIs) to a sequence of characters from the Universal Character Set [DS05]. A URI in general is a compact string of characters for identifying an abstract or physical resource [BLFM⁺98]. In our case we use the concept of IRIs to standardize the different service types among the agents, which is similar to the usage of performatives in ACLs, as described in Section 2.1.2. Another piece of information that is cached, are the costs. Those are not globally set, but rather a locally and for every agent individually defined metric that is used, to choose a service instance out of a group of fitting services. For example when multiple instances of the same service type are offered in the system (which is the case for the Material Properties service), an agent can define costs for services, in order to help him choose a specific service instance. The metric that is used by the Service Broker Agent and that can be seen in the cache in Listing 4.1 is the response time of an agent during the last service interface request in seconds. Other senseful definitions for the costs could be the mean response time of all service interface requests or the mean/last response time of the service requests individually for every service type and agent. An alternative to measuring the response times could also be quantifying the quality of a service. The quality of a service could be for example the content of data that a service result contains.

The agents manage their caches using the *cache_controller.py* script, which offers the following functions:

- **cache_services_from_address(path_to_cache, address)**
Given the *path_to_cache*, this function requests the service interface of an agent located at an *address* and saves the information of the offered services.
- **get_address_of_service(path_to_cache, service_iri)**
This function can be used by an agent to retrieve the route, which is necessary to access a service of a service type, which is defined by the *service_iri*. If multiple service instances of the correct service type are found in the cache, the function will return the instance with the minimal costs.
- **get_all_addresses_of_service(path_to_cache, service_iri)**
Similar to the previous function, this returns all routes to service instances of a required service type.

```

1  {
2    "http://0.0.0.0:10000": {
3      "services": [
4        {
5          "costs": 0.003372,
6          "description": "This service provides the forming plan
                           for a drive shaft.",

```

```

7      "iri": "http://www.semanticweb.org/max/ontologies/2021/
8          9/Services_Drive_Shaft#Forming_Plan",
9      "route": "http://localhost:10000/forming_plan",
10     "title": "Forming Plan"
11 },
12 {
13     "costs": 0.003372,
14     "description": "This service provides information about
15                     the properties of a specific material.",
16     "iri": "http://www.semanticweb.org/max/ontologies/2021/
17         9/Services_Drive_Shaft#Material_Properties",
18     "route": "http://localhost:10000/material_properties",
19     "title": "Material Properties"
20 }
21 ],
22 },

```

Listing 4.1: Parts of the service cache of the Service Broker Agent. Here one can see the cached information about the from the IBF Agent offered services.

When an agent has discovered another agent, which offers the required service, it can access the services via HTTP requests. For that, it prepares the request by filling out the input part of the service interface and then sends a post request to the route address. The service result is embedded in additional metadata, which is structured in the service ontology shown in Figure 3.3, to allow contextualization of the service result and potential process control, which we will discuss in the next section.

4.2.3 Process Control Module

Since we use a distributed approach to process control, all agents may implement different methods for process control. These methods do not have to be bundled up into a singular module, but we use the expression to refer to all those methods, which have functionality that is used for process control. Since the room for errors is very high and unpredictable in reality, we limit our use case to specific types of failures:

1. **Material too expensive:** In this case, the production plan agent was not able to find a material with costs lower than the amount that was specified by the user.
2. **No material found:** Here the production plan agent was not able to find a material, with the minimal material properties, that were initially defined by the user request. Without a material, the request of the subplans and therefore the creation of the complete production plan is not possible.
3. **Wrong force during forging process:** This is an issue being a result of the difference between the material, which the workpiece is made of, and the actual workpiece itself. When the properties of the workpiece deviate in a specific way from the theoretical properties of the material and the information provided in the network, the IBF Agent detects an unexpected force during the forging process and needs to address this issue.

The first case is detected and handled by the production plan agent. As visualized in Figure 4.1, the agent crawls the material properties and retail data from the other agents,

filters materials, which do not fit the minimal and necessary material requirements and then sorts the remaining material candidates according to the price. The production plan agent always chooses the cheapest material and continues to create a complete production plan, even if the material costs are higher than the costs, which were set by the user. This makes sense since the maximal costs from the user request are not a hard condition for the final plan and the more expensive alternative should be presented to the user. In addition to that, it should directly be indicated to the user that the shown production plan fits to an altered version of the user input. For that, the agent uses the service parameters which are also defined in the service ontology, as shown in Figure 3.3. Specifically, the production plan agent adds *altered input* information, indicating which input was altered to what value. The Service Broker Agent can then use this service result metadata and add an additional warning node displaying this information.

The second case is also detected and handled by the production plan agent. Since the minimal material properties that are specified by the user are hard constraints and need to be fulfilled, it is not possible to search for the next best material. Therefore, it is not possible to create a production plan, when no material is found. The process control then consists of the production plan agent informing the user through the Service Broker Agent. In this case, it returns a textual answer of how no material could be found with the necessary properties. The Service Broker Agent then shows the textual answer instead of a directed graph visualizing a production plan.

In the last and most complex case, the IBF Agent detects and handles the problem. Since the unexpected force during the forging process can have different causes, the agent needs to clarify that the material has properties, which deviate from the theoretical properties that are known in the system. For that, after detecting the unusual force, it checks the geometry of the workpiece, which can be done locally. After that, the agent needs to verify the core temperature of the workpiece during the heating process. Since the agent only holds information about the forging process, it needs to retrieve the information about the heat treatment process from the IWM Agent. In order to get the heat treatment data, the IBF Agent requests the heat treatment plan service of the IWM Agent. Then, it can check the core temperature of the workpiece and, if the temperature matches the expected temperature, knows that the unexpected force is caused by the workpiece's real material properties deviating from the theoretical ones. After identifying the cause of the issue, the agent can resolve it locally by returning an altered forging plan, so that the workpiece can still be used. This whole process is also visualized in the sequence diagram in Figure 4.2.

4.3 Specific Agent Services

Material Properties Service The *Material Properties* service provides information about the properties of a material, which are in our use case the yield strength, weldability, machinability and heat treatability.

The agents that offer this service own partial data about the properties in their local FactDAGs. When another agent requests this service, the local agent retrieves the fact containing the information about the properties from the storage. Then, it uses the **IRI** of the material, which is required as the service input, to search for the data that describes the given material. It then returns all the locally stored information about the specified material. Since the agents cover different productional areas, the returned

<i>Title</i>	Material Properties
<i>Description</i>	This service provides information about the properties of a specific material.
<i>Inputs</i>	Material IRI
<i>FactDAG Process</i>	provideMaterialProperties
<i>Instances, Agents</i>	IBF Agent (localhost:10000), IWM Agent (localhost:10001), WZL Agent (localhost:10002)

Figure 4.6: Summary of the Material Properties service. The FactDAG process is the name of the activity associated with the agent that uses or generates the fact, as shown in Figure 4.4.

information can be for example the yield strength and weldability for the IBF Agent and the heat treatability for the IWM Agent. Which service instances exist in the system, i.e., which agents offer the Material Properties service are listed in the service summary in Figure 4.6. When the material is unknown to the agent, it returns a textual warning to the requesting agent. Alternatively, the agent could compare the input material syntactically to the materials, that are known to him, and return information about the closest material. The requesting agent could be informed about this process using the altered input parameter specified in the service ontology.

Material Retail Service The *Material Retail* service provides information about the costs of a material. It works similar to the Material Properties service, but it exclusively provides the costs of a given material and it is only offered by the Steel Retailer Agent, as shown in Figure 4.7.

<i>Title</i>	Material Retail
<i>Description</i>	This service provides the retail information for a specific material.
<i>Inputs</i>	Material IRI
<i>FactDAG Process</i>	provideMaterialProperties
<i>Instances, Agents</i>	Steel Retailer Agent (localhost:10003)

Figure 4.7: Summary of the Material Retail service.

Material Buy Workpiece Service The *Material Buy Workpiece* service is another to the Steel Retailer agent exclusive service, which is used in order to buy a workpiece made of a certain material.

<i>Title</i>	Material Buy Workpiece
<i>Description</i>	This service provides the possibility to buy a workpiece of a certain material.
<i>Inputs</i>	Material IRI
<i>FactDAG Process</i>	provideWorkpieceInformation
<i>Instances, Agents</i>	Steel Retailer Agent (localhost:10003)

Figure 4.8: Summary of the Material Buy Workpiece service.

When this service is requested, the agent loads the fact that stores the data about the workpieces in stock and returns the information about a workpiece, which is made out of the requested material. The workpiece information consists of an internal **Id**, the **Width**, **Height** and **Length** as well as the **Material** of a workpiece.

Subplan Services There are three services, which are providing the plans for the subprocesses of the creation of the drive shaft. These services are the *Forming Plan* service, the *Heat Treatment Plan* service and the *Machining Plan* service.

<i>Title</i>	Forming Plan — Heat Treatment Plan — Machining Plan
<i>Description</i>	This service provides the (forming — heat treatment — machining) plan for a drive shaft.
<i>Inputs</i>	Material IRI, Workpiece
<i>FactDAG Process</i>	provideFormingPlan — provideHeatTreatmentPlan — provideMachiningPlan
<i>Instances, Agents</i>	IBF Agent (localhost:10000) — IWM Agent (localhost:10001) — WZL Agent (localhost:10002)

Figure 4.9: Summary of the plan services, which are providing the production plans for the subprocesses.

The plans are structured using the IBF and the P-Plan ontology, which are described in Section 3.2.1. The plans are saved in JSON files, where the plan steps are grouped by type and contain information about the input and output variables of a step, as well as information about which substep is the proceeding step. The types of substeps, which can be contained in a plan, are structured in the IBF Ontology (see Figure 3.2): There are the three general substeps: a **Forming** step with a more specific **Rolling Pass** step, a **Heat Treatment** step and **Machining** step. The substeps can later be further detailed by extending the IBF ontology in general or, as mentioned in Section 2.1, by adding local ontologies that extend the IBF ontology (core ontology). Every rolling pass step has **Rolling Pass Properties** as the input, which describe how the rolling pass is to be executed. The heat treatment subprocess has no input variables, but the **Temperature** of the workpiece as the output. The final step, the machining process, has neither input nor output variables. How the plans are later visualized by the Service Broker Agent is further detailed in Section 4.4.

<i>Title</i>	Total Plan
<i>Description</i>	This service provides a total production plan for a drive shaft.
<i>Inputs</i>	Heat Treatable, Machinability, Weldability, Yield Strength, Cost per Kilogram
<i>FactDAG Process</i>	planServiceResult
<i>Instances, Agents</i>	Production Plan Agent (localhost:5050)

Figure 4.10: Summary of the Total Plan service.

Total Plan Service The *Total Plan* service can be requested by a user, in order to retrieve the complete production plan for a drive shaft, which consists of a material with specific properties. For that, the production plan agent waits for a user request, as shown in Listing 4.2. When the user requests the service, the predicate *service_is_requested* is set, and therefore the procedure *handle_request* is called.

```

1  proc(control, [
2      while(true,
3          if(service_is_requested, handle_request, wait)
4      )
5  ]).

```

Listing 4.2: The control loop of the production plan agent.

In Listing 4.3 one can see the first action of the service process, which is the finding of a material. For that, the production plan agent requests the material properties from the other agents and, if it finds a material, the variable *Material* is substituted with the IRI of the material. Otherwise, when no material can be found, *Material* gets evaluated to *false*. If that is the case, the agent directly returns a notification. In the other case, when a material was found, the agent next executes the procedure *return_plan* shown in Listing 4.4, which uses the material to request the subplans and combines them into a complete production plan.

```

1  proc(handle_request, [
2      find_material(Material, Material_Costs),
3
4      if(Material == false,
5          return_notification,
6          return_plan(Material, Material_Costs)
7      )
8  ]).

```

Listing 4.3: The *handle_request* procedure of the production plan agent.

Since the material, which was chosen, can still have a higher price than what the user specified, the first step in the *return_plan* procedure is to check if that is the case. Depending on the outcome of this test, the agent prepares a result template, which normally contains no additional information. If the price is too high, a warning is added to the result template, which indicates to the user that the price was higher than requested and what the altered costs for the material are. After that, the agent orders a workpiece using the corresponding service from the Steel Retailer Agent, so that it can request the subplans. The request of the subplans can be done concurrently, as indicated by the usage of the predicate *conc*, which speeds up the process of requesting the subplans compared to sequential requests.

```

1  proc(return_plan(Material, Material_Costs), [
2      check_material_price(Material_Costs, Price_Too_High),
3      if(Price_Too_High,
4          result_template_warning(Material_Costs, Result_Template),
5          result_template_normal(Result_Template)
6      ),

```

```

7   order_workpiece(Material, Workpiece),
8   conc(
9       get_forming_plan(Material, Workpiece, Plan_Forming),
10      conc(
11          get_heat_treatment_plan(Material, Workpiece,
12                                  Plan_Heat_Treatment),
13          get_machining_plan(Material, Workpiece, Plan_Machining
14                              )
15      ),
16      create_plan([Plan_Forming, Plan_Heat_Treatment, Plan_Machining
17                  ], Workpiece, Plan_Total),
18      save_result(Plan_Total, Result_Template)
19  ]).

```

Listing 4.4: The *return_plan* procedure of the production plan agent.

When the production plan agent receives the answers from the other agents, which provide the subplans, it can create the complete production plan by combining the subplans and adding the workpiece as the input for the first step in the plan. Then the final plan gets saved as a fact in the local FactDAG and returned to the Service Broker Agent, which translates the plan into a directed graph.

4.4 User Interface

The user interface is realized using a local agent, the **Service Broker Agent**. It is responsible for providing the user with the correct service and it presents the service result to the user. The service result is in our case a plan visualized as a directed graph with additional information, or simply a textual answer. The agent offers a web interface, which is located at **localhost:11000** and which provides a google-like service search, as shown in Figure 4.11. The user can describe the service that he wants to request and submit a search request to the Service Broker Agent, which has its own service cache. When the search request is received, the agent searches for a service that has a title or description similar to the user's service description. It then lists all fitting services as well as additional information, such as the services' names, locations, descriptions and costs. The costs can once again be arbitrarily defined, and are reflecting the duration of the last service interface request in this case. The results can be additionally filtered by the costs, or one can simply tell the agent to show all in the network offered services.

Visualization of the Service Result Other than textual service results, which are simply shown directly to the user, plans are translated into directed graphs according to the P-Plan ontology, as seen in Figure 4.12. Every plan step is translated into a node in the graph, where the directed edges indicate the preceding steps. Since the plan consists of multiple subplans, every step belongs to a different subprocess, which is indicated through the different colors of the step nodes. The yellow nodes are visualizing the variables of the plan, with input and output variables. These are in our case the input roll pass properties for the rolling passes and the output temperature of the heat treatment step. The node on the bottom left is an altered input warning notification, which gets shown, when a material was found but has a higher cost than initially requested

WWL Agents

Service

Describe the name or functionality of the service.

Maximal Costs

☐ Show all services

If you newly installed the system, you need to initialize the FactDAG storages of the agents once.

Results

Service Name	Location	Description	Costs	Request
Machining Plan	http://0.0.0.0:10002	This service provides the forming plan for a drive shaft.	3.71	Request
Material Properties	http://0.0.0.0:10002	This service provides informations about the properties of a specific material.	3.71	Request
Material Buy Workpiece	http://0.0.0.0:10003	This service provides the possibility to buy a workpiece of a certain material.	2.85	Request

Figure 4.11: The user interface, where the user can search for a service.

by the user. When clicking on a node, additional information is shown, which can be seen in Listing 4.5. In this example, one can see the information of the fifth rolling pass step. Specifically, the input and output variables, the index of the preceding step as well as to which subprocess this step belongs are displayed. In addition to the plan information, some data about the service and provenance of the result step is shown under *Provenance_Information*. One can see that the data of this step was provided by a service provided by the *Service_Authority*, which is the IBF Agent in the given example, and that another authority was involved in the service execution. In this case, the involved authority is the IWM Agent, since the IBF Agent adjusted its forging plan and therefore requested information about the temperature, which is provided by the IWM Agent. Data provided by a service authority is always associated with an authority, against which the involved authorities can also be empty, which is the case when an agent executes its service without needing the help of another agent.

```

1  {
2    "@context": {
3      "dss": "http://www.semanticweb.org/max/ontologies/2021/9/
        Services_Drive_Shift#",
4      "ibf": "http://www.semanticweb.org/mohamadalomari/
        ontologies/2021/6/IBF1107#",
5      "plan": "http://www.semanticweb.org/max/ontologies/2021/10
        /Plan_Drive_Shift#"
6    },
7    "plan:hasInputVar": {
8      "ibf:Rolling_Pass_Properties": {
9        "ibf:b_0": 105,
10       "ibf:b_1": 115,
11       "ibf:h_0": 142,
12       "ibf:h_1": 115,
13       "ibf:l_0": 758,
14       "ibf:l_1": 857

```



```

15     }
16   },
17   "plan:hasOutputVar": [],
18   "plan:isPrecededBy": {
19     "plan:Rolling_Pass": 3
20   },
21   "plan:isStepOfPlan": "plan#Forming",
22
23   "dss:Provenance_Information": {
24     "dss:Involved_Authorities": "http:localhost:10001",
25     "dss:Service_Authority": "http://localhost:10000"
26   }
27 }

```

Listing 4.5: The information that is shown when clicking on a node on the visualized production plan.

Starting the System Since all of our agents run in **docker** [Mer14] containers, which are software units packaging up code and all its dependencies, the system runs reliably on different computing environments. The system can simply be started using **docker-compose**. The docker images can be built and the system started by running the following commands:

```

$ docker-compose build
$ docker-compose up -d

```

Because initially the FactDAG is empty, the web interface provides a button for initializing the local data storage for every agent. This initialization process needs to be done, in order for the system to work.

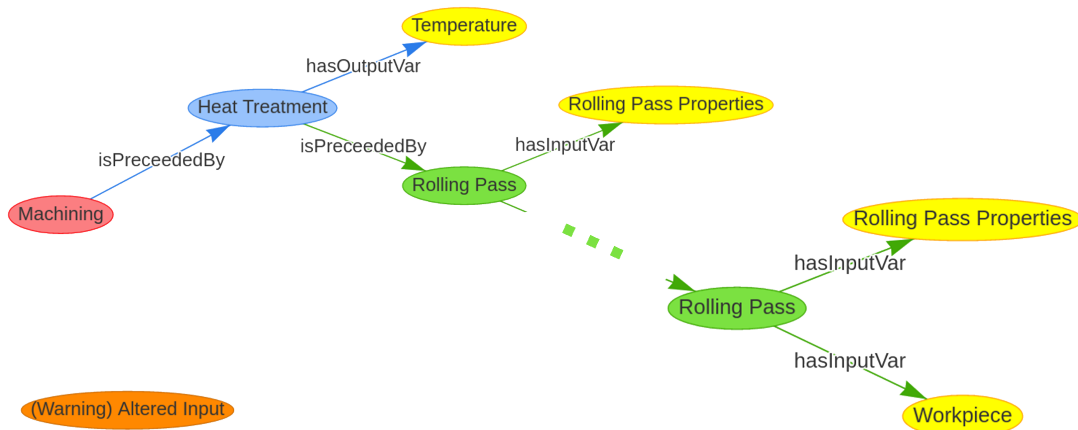


Figure 4.12: The visualization of the plan, which is presented to the user by the Service Broker Agent.

5 Discussion and Future Work

We have described the theoretical components as well as the practical implementation of the multi-agent system. Now we are going to discuss the advantages and disadvantages of the approaches that we used. Specifically, we examine the service quality and efficiency of the system as well as the resulting plan. After that, we describe the strengths and weaknesses of the process control and service discovery used by the agents, as well as the open challenges that have to be addressed in future work.

5.1 Service Quality and Efficiency of the Multi-Agent System

Service Result and Data Provenance Visualization The service result is presented as a directed graph (see Figure 4.12) to visualize the production plan and provenance information is textually shown when clicking on a node. The P-Plan ontology [DG14] and the parameters of the service ontology are a solid base for structuring and later visualizing productional plans and the provenance information of data used. However, the provenance information is currently only shown textually and lacks precision. The system provides the user with the authorities, which provide the service or were involved in the service execution, but it is not clear how the other agents were involved in the service process or what data they provided for its successful execution.

The service ontology, which describes the service parameters containing provenance information, needs to be extended, to show more detailed information about the provenance of data. In addition to that, the provenance information could be visualized by adding further nodes representing services executed by agents to the final graph, which is presented to the user.

FactDAG Data Storage with factlib.js The data storages are structured according to the FactDAG model by Gleim et al. and implemented using factlib.js, which was developed by Liam Tirpitz [Tir20]. The Typescript library factlib.js allows developers of multi-agent systems to easily implement storage systems that store data as facts and fulfill all four FAIR principles. But as already mentioned in Section 3.1, it only addresses three out of the five FactDAG requirements [LG⁺20]. Data sovereignty and stakeholder confidentiality and minimalism are in our current implementation with factlib.js not satisfied and need to be solved in the future. One step towards stakeholder confidentiality would be to use authenticated encryption for data-sharing services. Since our services are accessible via HTTP requests, we should use the Transport Layer Security (TLS) protocol [Tho00] for sending and receiving service data. Additionally, the service identification process, which is done via UDP, should also be encrypted to keep the addresses of agents that participate in the system secured from unauthorized access. The Datagram Transport Layer Security (DTLS) protocol [RM12] provides communication privacy for datagram protocols and could be used to secure the data shared during the service discovery processes in the system. Lastly, it should also be possible to only enable a service for certain authorities or companies [Bal20].

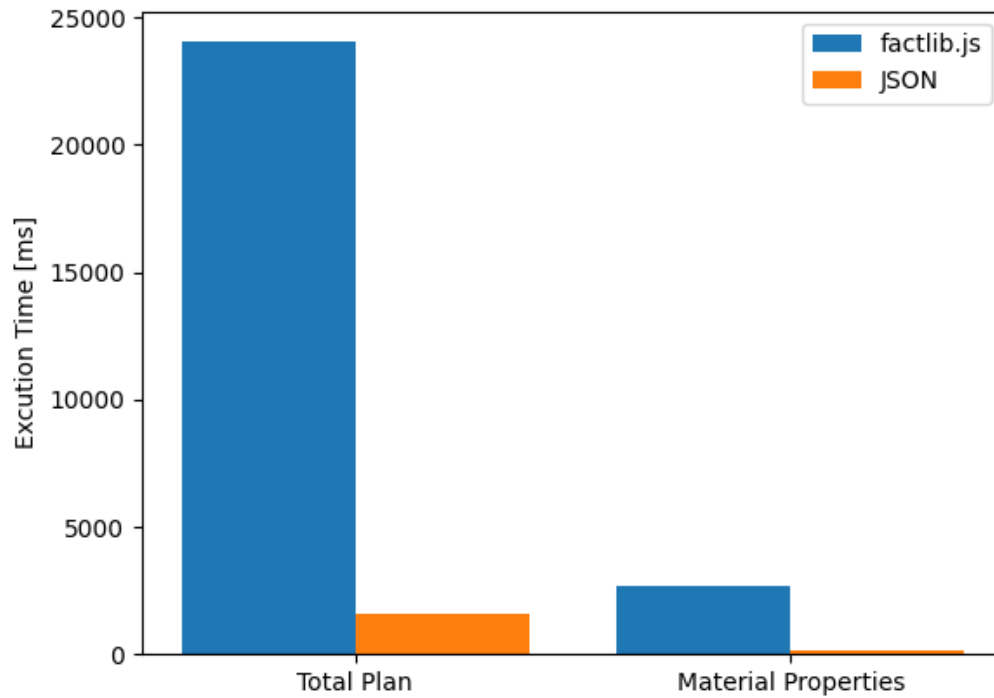


Figure 5.1: Comparison of the execution time for the Total Plan and Material Properties service, once with `factlib.js` and once from loading from a local JSON file. The Total Plan service is provided by the production plan agent and the Material Properties service instance that was tested is the instance provided by the IBF Agent.

Another problem is the access speed to the data storage when using `factlib.js`. Every read and write operation takes several seconds, making the service time of the total plan service, which requires several retrievals of facts from local FactDAGs, rather long. This can be seen in the graph shown in Figure 5.1, which puts the service execution times for the Total Plan and Material Properties services in context with the usage of `factlib.js`. When we compare the execution times of the services, when using `factlib.js`, with the total amount of data storage hits, which are performed during the service execution and shown in Figure 5.2a, we see a positive correlation between service execution time and the number of data storage hits. In Figure 5.2b one can see that the `factlib.js` data storage operations take about 93.4% of the whole Total Plan service execution time, leaving only 6.6% of execution time to other computations.

One reason for this bottleneck is `factlib.js` being a Typescript library, meaning that before execution the used scripts have to be compiled to JavaScript, which currently has to be done during runtime. In addition to the on-runtime compilation, the execution time of the provided functions themselves is not very performant and further reinforces this bottleneck. It is clear that the service execution time when using local JSON files is an unreachable optimum, since that way the metadata, which is required for the FactDAG, is not added. However, these comparisons still show that the performance decrease is

disproportionate to the additional operations, which are needed in order to transform data into facts. The overhead caused by structuring data as facts is especially critical since the production plan is created using many different microservices, where a rather big amount of facts, which all contain small amounts of data, are retrieved from the local FactDAGs. The `factlib.js` library should be altered in order to enable compiling the TypeScript files down to JavaScript before runtime and the general performance of the provided functionalities needs to be improved.

The advantage of this structure is however, that we indeed distribute the necessary computation power across all agents, and split up the task by distributing subgoals to different expert agents (i.e., the IBF Agent for the forming process, the IWM Agent for the heat treatment process, etc.). This also allows further parallelization and speeding up of processes, which is for example done by the production plan agent, which concurrently requests the subplans as shown in Listing 4.3, and therefore divides the goal of creating a complete production plan in several smaller tasks for expert agents.

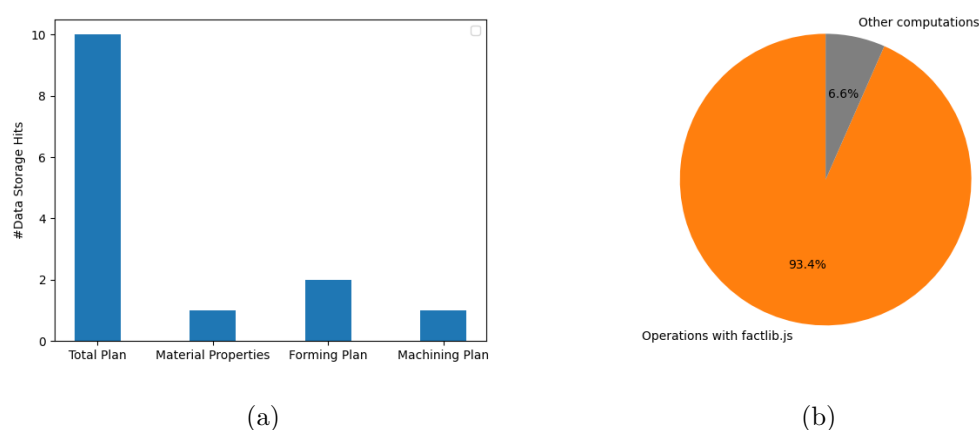


Figure 5.2: Figure a) shows the number of data storage hits during the execution of the different service types. Figure b) shows the fraction of the execution time of `factlib.js` operations in comparison with the rest of the computations that are performed during the Total Plan service.

User Interface The Service Broker Agent provides the user with a web interface, where he can search for a service. As one can see in Figure 4.11, the interface lists all services, that fit to the user’s description in the input field, and shows additional information. Therefore, it is a clear access point for a user to all World Wide Lab agents. The interface also enables the user to filter services according to some kind of costs, which help him decide which instance he should request out of several instances of the same service type. The costs currently represent the service interface response time of an agent, which is a very rudimentary metric. In addition to that, it is not explained to the user, how these costs are calculated. A better metric for the costs could be the mean execution time of a service, which could be one of many metrics the user could choose from.

After selecting a service, the user is presented with the dynamically created service interface page, which can be seen for the Total Plan service in Figure 5.3. As long as a service implements the standardized service interface and service discovery, it can be seen by the Service Broker Agent, which can then dynamically list the service and create the

service input page, based on the inputs described in the interface. However, the input page is currently only suited for numerical and simple string values, like they are for example required for the Total Plan service and the Material Properties services (see Section 4.3). The services that provide the plans for the different subprocesses require a workpiece as the input, which is represented by a complex JSON object, which currently needs to be entered via a simple input field. In the future the web interface could offer a tool that simplifies the creation of complex JSON inputs.

WWL Agent: <http://0.0.0.0:5050>

Service: Total Plan

Cost per Kilogram

Ontology: http://www.semanticweb.org/mohamadalomari/ontologies/2021/6/IBF1107#Cost_per_Kilogram

Heat Treatable

Ontology: http://www.semanticweb.org/mohamadalomari/ontologies/2021/6/IBF1107#Heat_Treatable

Machinability

Ontology: <http://www.semanticweb.org/mohamadalomari/ontologies/2021/6/IBF1107#Machinability>

Weldability

Ontology: <http://www.semanticweb.org/mohamadalomari/ontologies/2021/6/IBF1107#Weldability>

Yield Strength

Ontology: http://www.semanticweb.org/mohamadalomari/ontologies/2021/6/IBF1107#Yield_Strength

[Request](#)

Figure 5.3: The dynamically created input page for the Total Plan service. The page shows the authority that offers the service (*localhost:5050*), the service name (*Total Plan*) and the ontology, with which the input is associated (here the *IBF ontology*).

5.2 Process Control and Service Discovery

We implemented a distributed and cooperative approach to process control, where there is no global view of the system. However, issues are handled by the agents, that locally detect the faults. Currently, we handle three cases: the chosen material being too expensive, the production plan agent not being able to find a material with sufficient properties and a wrong force getting detected during the forging process. Sometimes, an agent needs additional information in order to handle an issue, which it can retrieve by communicating with the other agents. For example, the IBF Agent requests the temperature of the workpiece from the IWM Agent in order to exclude an unwanted temperature change as the cause of the unexpected force, which was detected during the forging process. Other problematic cases are currently not covered and can therefore not be handled by the system. In contrast to the advantage of distributing process control tasks across the agents, a disadvantage is that there exists no global overview of errors that the system can handle. Since at the moment some agents offer unique services, an issue that the unique agent could not handle would cause the entire Total Plan service to fail. That means that without real alternatives to every agent the sys-

tem is not completely distributed and still depends on single service instances and agents.

The service discovery is realized by every agent having its own service cache and a standardized multicast communication channel. Due to the approach being distributed, the service discovery does not depend on single or distinct yellow page services, but every agent saves the information about the agents and services, that he needs. Compared to distinct service discovery agents, this approach causes a memory overhead, since the same service information needs to be cached by all the agents, that need the service. The worst-case storage overhead, where every agent caches the information of every other agent in the system, would be $n \cdot s$, where n is the number of agents and s is the total amount of memory needed to store the service information of all agents once. In addition to that, every agent needs to request the service interface of all agents, which causes a communication overhead that theoretically scales quadratically to the number of agents in the network. One improvement for the communication overhead could be extra service discovery services that enable agents to share the content of their current service caches with other agents. That way the amount of agent interface requests could be reduced, while the service discovery itself would still be distributed and robust against failures of single nodes.

6 Conclusion

The goal of this thesis was to create a multi-agent system focusing on the areas of service discovery and process control. The MAS builds on an existing implementation by Nikola Balog [Bal20], which addresses the issue of semantic data interoperability mainly through standardization. All agents agree on a core ontology, that structures information about the production processes, but an agent can extend the core ontology locally and arbitrarily. Data is stored according to the FactDAG model [LG⁺20] to keep track of the provenance of data, which enables the user to evaluate the quality of the data used by the agents.

Our implementation further refines the approach for semantic interoperability by creating a service ontology with additional parameters that allow the agents to not only return the result of a service, but also metadata describing its execution. In addition to semantic interoperability, we also examined the issue of process control, where we implemented a distributed and cooperative approach. Every agent tries to resolve issues locally and if required with the help of the other agents. We limited the process control of our use case to three different issues, which all are handled either locally by the agent that detects them, by the agent with the information provided by another agent, or by properly communicating the problem to the user, if the issue can not be handled automatically. In order for the different agents to communicate with each other, agent identification and service discovery are required. Instead of specific yellow page services, which are responsible for conveying between different agents, every agent has its own local service cache and uses a standardized communication channel for broadcasting messages. That way, the service discovery of the system does not depend on single and distinct service discovery units, but each agent caches the needed service information locally and independently. All services of the decentralized system are available for the user through a single web interface, which is provided by a local agent. The web interface offers a search functionality, where the user can find the service that they need. The service input parameters can be entered at the service input page, which gets dynamically created based on the standardized agent interface, that describes the services of an agent with its inputs. We implemented the system in the context of the automotive industry. Specifically, we aimed at implementing the MAS with the goal to create a production plan for a drive shaft, a component of vehicles that translates the torque from the gearbox to the wheels. The different agents of the system all have their expert areas and work together, in order to combine subplans into a complete production plan. The resulting plan is then shown as a directed graph in the web interface, where the user is additionally presented with information about the provenance of the data, which was used during the service execution. We also discussed issues, which need to be addressed in future work, especially the performance of the library `factlib.js` could hinder future implementations to perform well.

Bibliography

- [AB17] C. Schindhelm A. Bröring, S. Schmid. Enabling iot ecosystems through platform interoperability. *IEEE Software*, 34:54–61, 2017. (Cited on page 4.)
- [Arn99] Ken Arnold. The jini architecture: Dynamic services in a flexible network. In *Proceedings of the 36th Annual ACM/IEEE Design Automation Conference, DAC '99*, page 157–162, New York, NY, USA, 1999. Association for Computing Machinery. (Cited on page 7.)
- [Bal20] Nikola Balog. Multi-agent system for supply chain management in the textile industry. 2020. (Cited on pages 2, 4, 10, 33 and 38.)
- [BCD02] F. Dignum B. Chaib-Draa. Trends in agent communication language. *Computational Intelligence*, 18, 2002. (Cited on page 5.)
- [BLFM⁺98] Tim Berners-Lee, Roy Fielding, Larry Masinter, et al. Uniform resource identifiers (uri): Generic syntax, 1998. (Cited on page 24.)
- [Cam02] Celeste Campo. Directory facilitator and service discovery agent. FIPA Document Repository, 2002. (Cited on page 7.)
- [CB02] Celeste Campo and Peter T. Breuer. Service discovery in pervasive multi-agent systems, 2002. (Cited on page 7.)
- [Col98] A. W. Colombo. *Development and Implementation of Hierarchical Control Structures of Flexible Production Systems Using High-Level Petri Nets*. Prof. Dr.-Ing. Klaus Feldmann, Prof. Dr.-Ing. Dr. h.c. Manfred Geiger, 1998. (Cited on page 2.)
- [Con04] W3C Consortium. Owl web ontology language overview. <https://www.w3.org/TR/2004/REC-owl-features-20040210/>, 2004. (Cited on page 11.)
- [Con20] W3C Consortium. Web of things (wot) thing description. <https://www.w3.org/TR/wot-thing-description/>, 2020. (Cited on page 12.)
- [DD91] H. Whorms D. Diltis, N. Boyd. The evolution of control architectures for automated manufacturing systems. *Journal of Manufacturing Systems*, 1(10):63–79, 1991. (Cited on page 8.)
- [DG14] Y. Gil D. Garijo. The p-plan ontology. <http://vocab.linkeddata.es/p-plan/index.html>, 2014. (Cited on pages 12 and 33.)

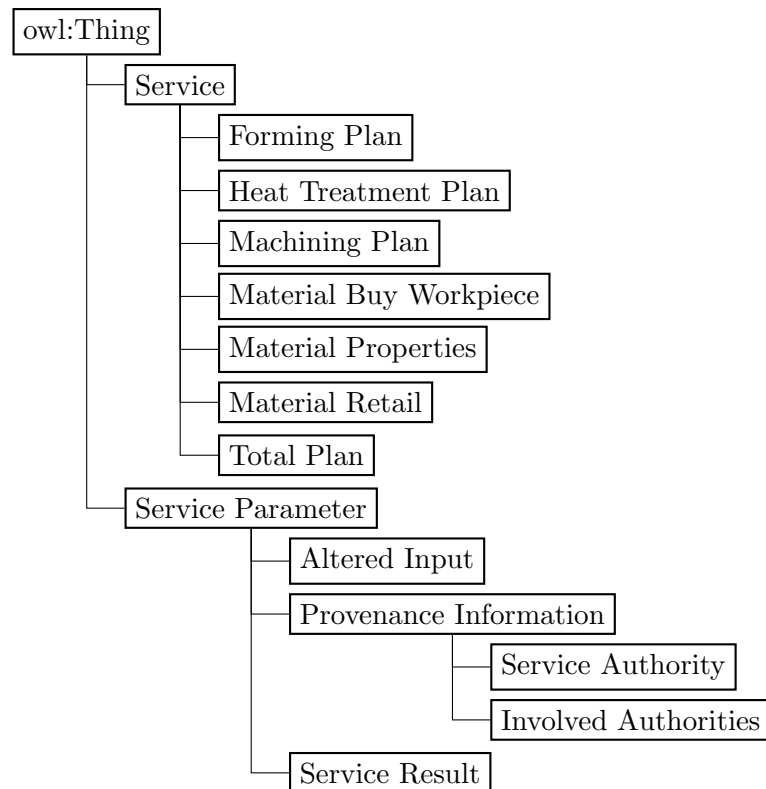
- [DM⁺07] D. McDermott D. Martin, M. Burstein et al. Bringing semantics to web services with owl-s. *World Wide Web*, 10, 2007. (Cited on page 7.)
- [DS05] Martin Dürst and Michel Suignard. Internationalized resource identifiers (iris). Technical report, RFC 3987, January, 2005. (Cited on pages 6 and 24.)
- [EdV09] V. Botti E. del Val, M. Rebollo. Semantic service discovery in mas using social networks, 2009. (Cited on page 6.)
- [FB99] G. Rimassa F. Belfemine, A. Poggi. Jade - a fipa-compliant agent framework. *Proceedings of PAAM*, 1999. (Cited on page 6.)
- [FB00] G. Rimassa F. Belfemine, A. Poggi. *Intelligent Agents VII Agent Theories Architectures and Languages*, chapter Developing Multi-agent Systems with JADE, pages 89–103. Springer, 2000. (Cited on page 6.)
- [FD16] J. Horkof F. Dalpiaz, X. Franch. istar 2.0 language guide. 2016. (Cited on page 3.)
- [FIP02] FIPA. Fipa agent management specification. <http://www.fipa.org/specs/fipa00023/SC00023J.html>, 1996-2002. (Cited on pages 6 and 22.)
- [GDG09] S. Sardina G. De Giacomo. *IndiGolog: A High-Level Programming Language for Embedded Reasoning Agents*, chapter IndiGolog: A High-Level Programming Language for Embedded Reasoning Agents, pages 31–72. Springer, 2009. (Cited on pages 15 and 16.)
- [HJL97] Y. Lespérance H. J. Levesque, R. Reiter. Golog: A logic programming language for dynamic domains. *The Journal of Logic Programming*, 1997. (Cited on page 16.)
- [KFS06] Matthias Klusch, Benedikt Fries, and Katia Sycara. Automated semantic web service discovery with owls-mx. In *Proceedings of the Fifth International Joint Conference on Autonomous Agents and Multiagent Systems*, AAMAS '06, page 915–922, New York, NY, USA, 2006. Association for Computing Machinery. (Cited on page 7.)
- [Lau05] Lee Min Lau. Towards data interoperability: Practical issues in terminology implementation and mapping. *Towards Data Interoperability: Practical Issues in Terminology Implementation and Mapping/AHIMA*, American Health Information Management Association, 2005. (Cited on page 4.)
- [Lei08] Paulo Leitão. Agent-based distributed manufacturing control: A state-of-the-art survey. *Engineering Applications of Artificial Intelligence*, pages 980–991, 2008. (Cited on pages 2 and 8.)
- [LG⁺20] Jan Pennekamp Lars Gleim et al. Factdag: Formalizing data interoperability in an internet of production. *IEEE Internet of Things Journal*, 7, 4, 2020. (Cited on pages 2, 10, 11, 33 and 38.)

- [LM06] S.R.T. Kumara L. Monostori, J. Váncza. Agent-based systems for manufacturing. *CIRP Annals*, 55, 2006. (Cited on page 5.)
- [McC63] John McCarthy. Situations, actions, and causal laws. *STANFORD UNIV CA DEPT OF COMPUTER SCIENCE*, 1963. (Cited on page 15.)
- [MDW⁺16] M. Dumontier M. D. Wilkinson et al. The fair guiding principles for scientific data management and stewardship. *Sci. Data*, 2016. (Cited on page 10.)
- [Mer14] Dirk Merkel. Docker: lightweight linux containers for consistent development and deployment. *Linux journal*, 2014(239):2, 2014. (Cited on page 32.)
- [ML20] Matthias Jarke Martin Liebenberg. *Advanced Information Systems Engineering*, chapter Information Systems Engineering with Digital Shadows: Concept and Case Studies, pages 70–84. 2020. (Cited on page 2.)
- [MS⁺18] S. Abeck M. Schneider, B. Hippchen et al. Enabling iot platform interoperability using a systematic development approach by example. *2018 Global Internet of Things Summit (GloTS)*, 2018. (Cited on page 4.)
- [PL06] Armando Walter Colombo Paulo Leitão. A formal specification approach for holonic control systems: The adacor case. *International Journal of Manufacturing Technology and Management*, 2006. (Cited on page 9.)
- [QLG06] Bing Qiao, Kecheng Liu, and Chris Guy. A multi-agent system for building control. In *2006 IEEE/WIC/ACM International Conference on Intelligent Agent Technology*, pages 653–659, 2006. (Cited on page 9.)
- [Rei01] Ray Reiter. On knowledge-based programming with sensing in the situation calculus. *ACM Trans. Comput. Logic*, 2(4):433–457, oct 2001. (Cited on page 15.)
- [RM12] Eric Rescorla and Nagendra Modadugu. Datagram transport layer security version 1.2. 2012. (Cited on page 33.)
- [SAM01] Honglei Zeng Sheila A. McIlraith, Tran Cao Son. Semantic web services. *IEEE intelligent systems*, 2(16):46–53, 2001. (Cited on pages 4 and 5.)
- [SFV07] Elder Rizzon Santos, Moser Silva Fagundes, and Rosa Maria Vicari. An ontology-based approach to interoperability for bayesian agents. In *Proceedings of the 6th international joint conference on Autonomous agents and multiagent systems*, pages 1–3, 2007. (Cited on page 4.)
- [SV11] S. Bhavsar S. Vaniya, B. Lad. A survey on agent communication languages. *2011 International Conference on Innovation*, 14, 2011. (Cited on pages 4 and 5.)
- [Tho00] Stephen Thomas. Ssl and tls essentials. *New Yourk*, (3), 2000. (Cited on page 33.)

- [Tir20] Liam Tirpitz. A distributed data revisioning system for the internet of production. 2020. (Cited on pages 11, 21, 22 and 33.)
- [Tom10] Iulian-Florin Toma. Contributions to the study of semantic interoperability in multi-agent environments - an ontology based approach. *Int. J. of Computers*, V(5), 2010. (Cited on page 4.)
- [Vei97] et al. Veizades, John. Service location protocol. RFC 2165, 1997. (Cited on page 7.)
- [Wal99] Jim Waldo. The jini architecture for network-centric computing. *Commun. ACM*, 42(7):76–82, jul 1999. (Cited on page 7.)
- [WD⁺09] Wakulicz-Deja et al. Hierarchical multi-agent system. *Recent Advances in Intelligent Information Systems*, 615(628), 2009. (Cited on page 9.)
- [WSTL12] Jan Wielemaker, Tom Schrijvers, Markus Triska, and Torbjörn Lager. SWI-Prolog. *Theory and Practice of Logic Programming*, 12(1-2):67–96, 2012. (Cited on page 16.)
- [XW⁺04] D.Q. Zhang X.H. Wang et al. Ontology based context modeling and reasoning using owl. *IEEE Annual Conference on Pervasive Computing and Communications Workshops*, 2004. (Cited on page 11.)
- [YYG99] T. Cai Y. Y. Goland. Simple service discovery protocol, technical report, 1999. (Cited on page 7.)

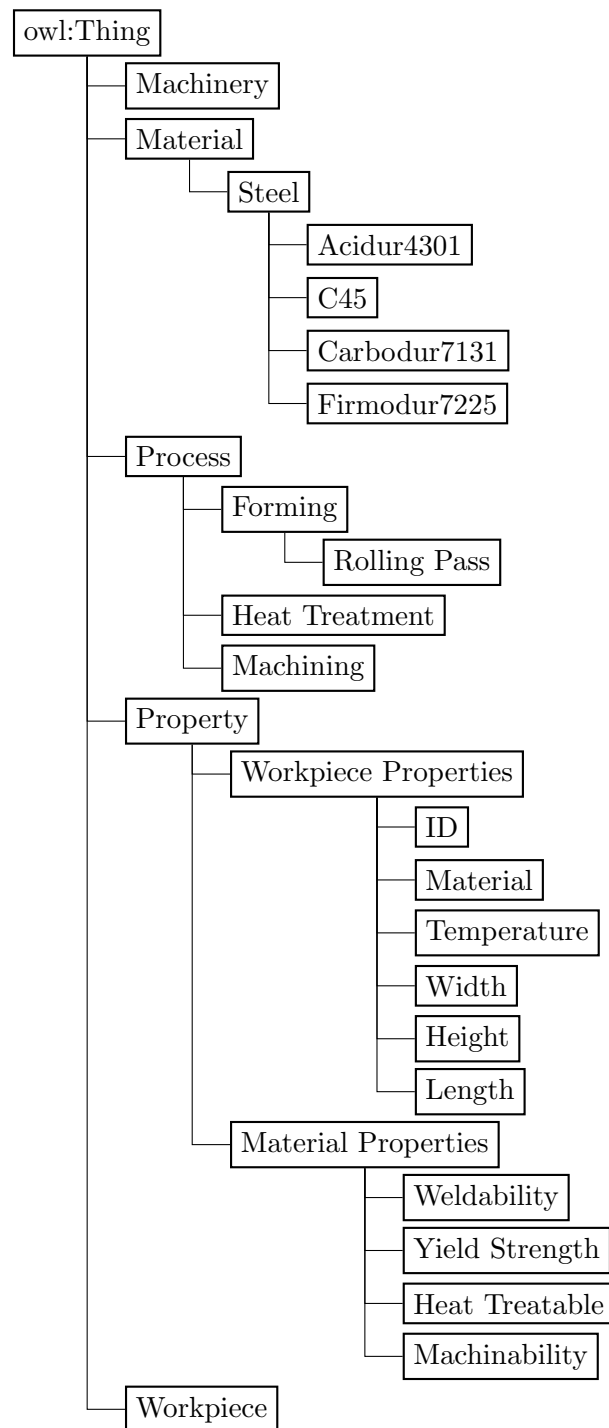
Appendices

A Service Ontology



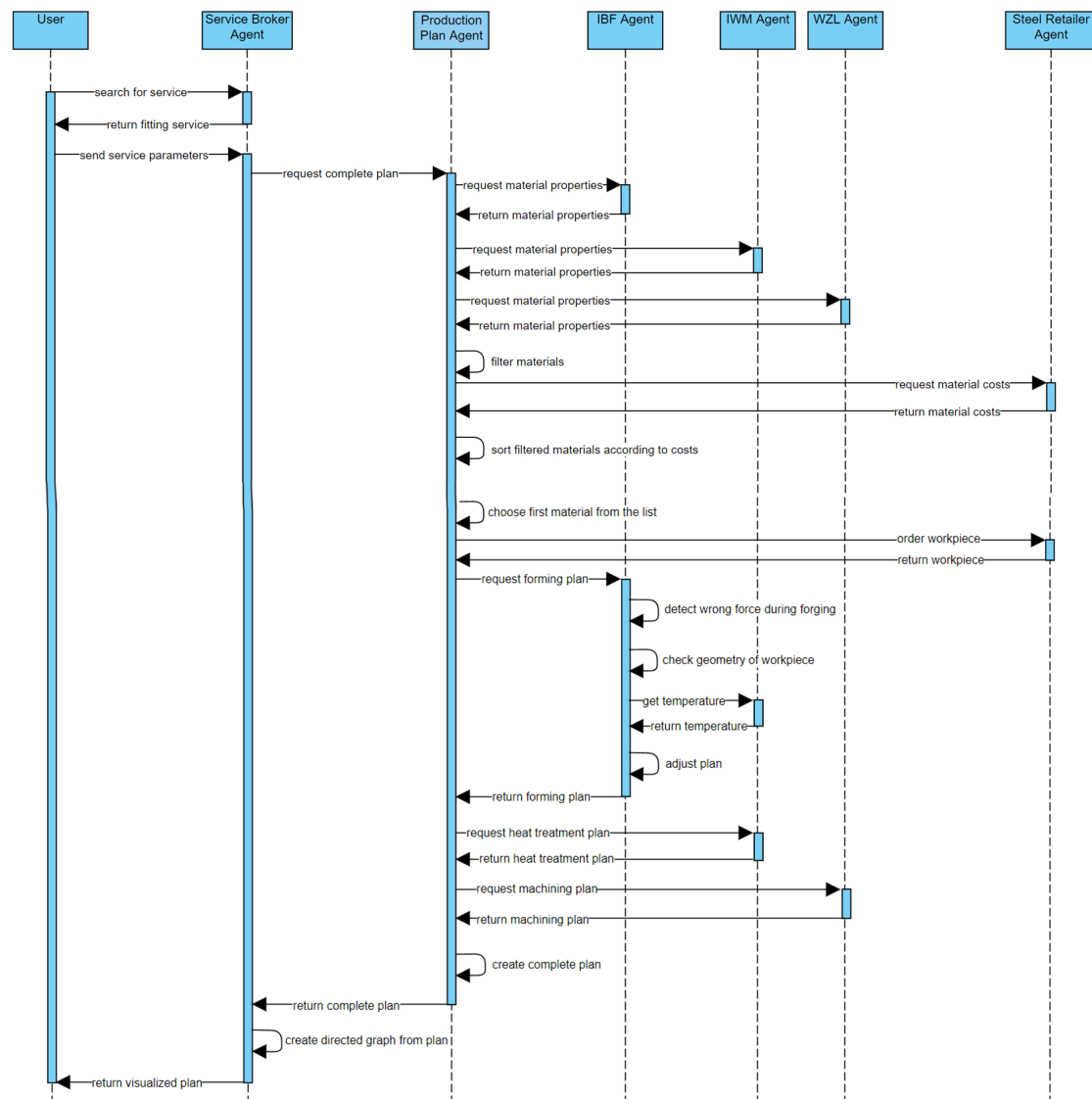
Extended overview of important parts of the service ontology.

B IBF Ontology



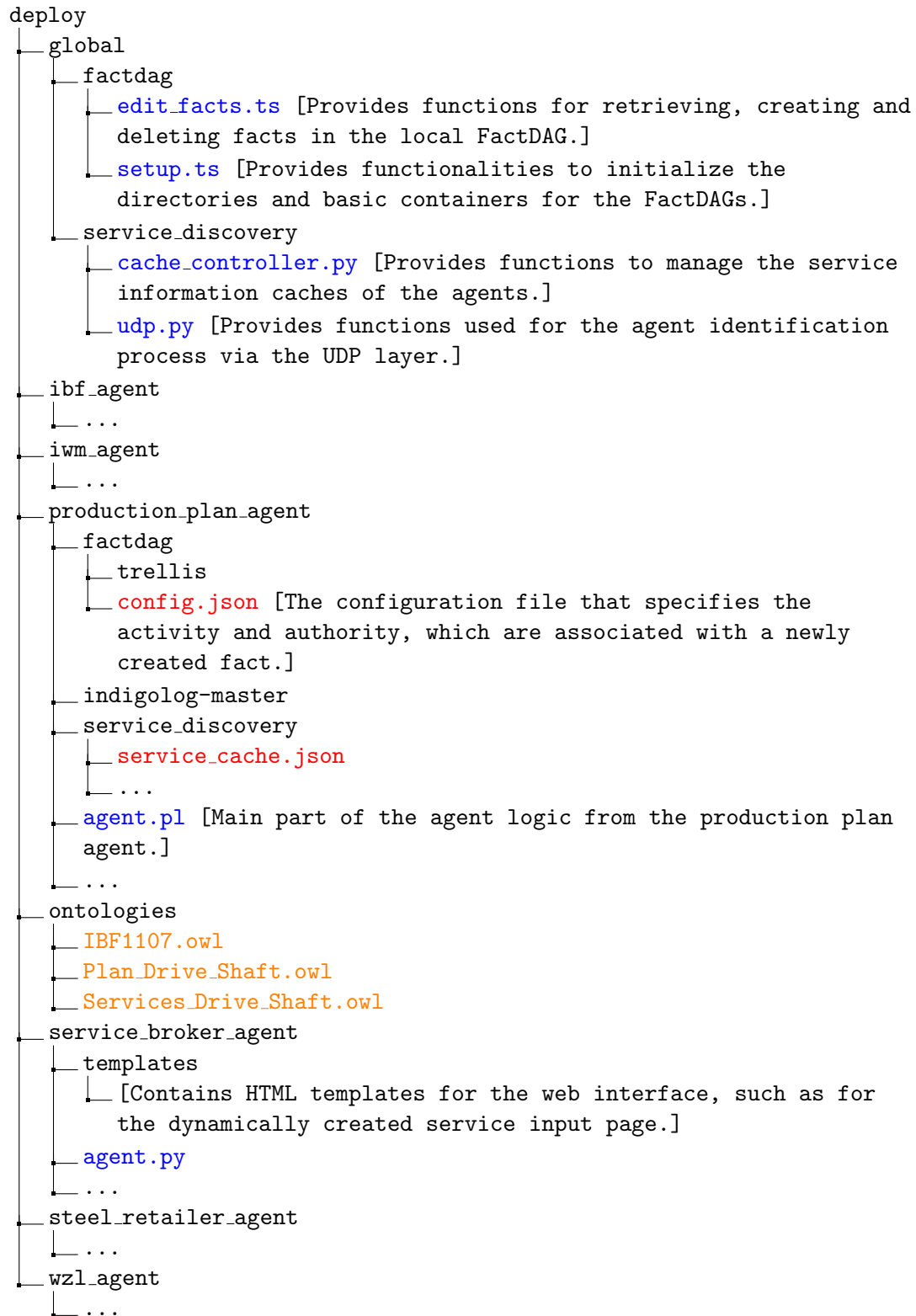
Extended overview of important parts of the ontology provided by the IBF.

C Sequence Diagram



Complete version of the sequence diagram showing the (inter)actions of the agents, which together create the production plan for the drive shaft.

D Folder Structure and Files



Folder structure and important files of the implemented multi-agent system.